

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ
MINH**
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

**TRIỂN KHAI HỆ THỐNG QUẢN LÝ TÀI LIỆU
BẢO MẬT DOCVAULT THEO QUY TRÌNH
DEVSECOPS**

Môn học: NT114 - Đồ án chuyên ngành
Sinh viên thực hiện: Huỳnh Lê Đại Thắng – 23521422
Nguyễn Trường Duy – _____
Giảng viên hướng dẫn: ThS. Lê Anh Tuấn

TP. Hồ Chí Minh, tháng 06 năm 2026

Lời cảm ơn

Trong quá trình thực hiện đề án chuyên ngành với đề tài “Triển khai hệ thống quản lý tài liệu bảo mật DocVault theo quy trình DevSecOps”, em đã có cơ hội vận dụng kiến thức về phát triển phần mềm, kiến trúc microservices, bảo mật ứng dụng và tự động hóa triển khai vào một hệ thống có tính thực tiễn cao. Đề tài không chỉ tập trung vào việc xây dựng một ứng dụng quản lý tài liệu, mà còn đặt trọng tâm vào cách đưa các kiểm soát bảo mật vào toàn bộ vòng đời phát triển.

Em xin gửi lời cảm ơn đến giảng viên hướng dẫn đã định hướng, góp ý và hỗ trợ em trong quá trình xác định phạm vi, thiết kế kiến trúc và hoàn thiện sản phẩm. Những góp ý trong quá trình thực hiện giúp em nhìn nhận rõ hơn mối liên hệ giữa yêu cầu nghiệp vụ, yêu cầu bảo mật và khả năng vận hành của hệ thống.

Em cũng xin cảm ơn nhà trường và khoa đã tạo điều kiện để em được thực hiện đề án theo hướng thực hành, qua đó rèn luyện kỹ năng xây dựng hệ thống, kiểm thử, triển khai và trình bày kết quả kỹ thuật một cách có hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo và sản phẩm vẫn có thể còn thiếu sót. Em mong nhận được góp ý để tiếp tục cải thiện hệ thống trong các giai đoạn sau.

Tóm tắt

DocVault là một hệ thống quản lý tài liệu bảo mật được xây dựng theo kiến trúc microservices, hướng đến bài toán quản lý vòng đời tài liệu trong môi trường có yêu cầu phân quyền, kiểm soát tải xuống, lưu vết audit và hỗ trợ compliance. Hệ thống cho phép người dùng tạo tài liệu, tải lên phiên bản file, gửi duyệt, phê duyệt, xuất bản, lưu trữ và truy vết các hoạt động liên quan. Các vai trò chính gồm Viewer, Editor, Approver, Compliance Officer và Admin. Mỗi vai trò có phạm vi truy cập khác nhau dựa trên RBAC, ACL, trạng thái tài liệu và mức phân loại bảo mật.

Về mặt ứng dụng, DocVault được triển khai với frontend Next.js, API Gateway và các backend service NestJS gồm metadata-service, document-service, workflow-service, audit-service và notification-service. Hệ thống sử dụng Keycloak cho xác thực, PostgreSQL cho metadata, MongoDB cho audit log và MinIO/S3 cho lưu trữ file. Các kiểm soát bảo mật trọng tâm bao gồm phân quyền theo vai trò, ACL deny-overrides, kiểm soát pre-view/download, grant token ngắn hạn, hash-chain audit, malware scanning, DLP, retention evidence và compliance evidence packet.

Về mặt DevSecOps, đề tài thiết kế và triển khai pipeline Jenkins nhằm tự động hóa các bước cài đặt phụ thuộc, kiểm thử, SAST, SCA, quét filesystem/image bằng Trivy, quét IaC bằng Checkov, validate Terraform, build Docker image, cập nhật GitOps branch, kiểm tra Argo CD health, smoke test sau triển khai và DAST bằng OWASP ZAP. Phần ứng dụng DocVault hiện đã gần hoàn thiện về runtime và bằng chứng kiểm thử; phần pipeline DevSecOps đang tiếp tục được hoàn thiện để tăng độ ổn định, khả năng lặp lại và chất lượng bằng chứng triển khai.

Từ khóa: DocVault, quản lý tài liệu, bảo mật ứng dụng, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Abstract

DocVault is a secure document management system designed around a microservices architecture and a DevSecOps-oriented delivery process. The system addresses document lifecycle management in an environment that requires role-based access control, controlled file access, auditability, and compliance evidence. Core workflows include document creation, file upload and versioning, review submission, approval, publication, archiving, preview/download authorization, audit inspection, and evidence export.

The application layer consists of a Next.js frontend, an API Gateway, and NestJS microservices, including metadata, document, workflow, audit, and notification services. Keycloak provides identity and access management, PostgreSQL stores document metadata, MongoDB stores audit events, and MinIO/S3 stores file objects. Security controls include RBAC, ACL with deny-overrides semantics, short-lived grant tokens, tamper-evident audit hash chains, malware scanning, DLP classification escalation, retention evidence, and compliance evidence packets.

The DevSecOps pipeline is implemented with Jenkins and integrates security checks throughout the build and deployment process. It includes dependency analysis, static application security testing, filesystem and image scanning, infrastructure-as-code scanning, Terraform validation, unit testing, Docker image build and scan, GitOps updates, Argo CD health checks, post-deploy smoke tests, and OWASP ZAP DAST. At the current stage, the application/runtime scope is nearly complete, while the DevSecOps pipeline is being refined to improve reliability, evidence quality, and deployment readiness.

Keywords: DocVault, document management, application security, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Mục lục

Lời cảm ơn	1
Tóm tắt	2
Abstract	3
Danh mục từ viết tắt	10
1 Tổng quan đề tài	12
1.1 Bối cảnh	12
1.2 Vấn đề cần giải quyết	13
1.3 Mục tiêu đề tài	13
1.4 Phạm vi thực hiện	14
1.5 Đóng góp của đề tài	15
1.6 Cấu trúc báo cáo	15
2 Cơ sở lý thuyết và công nghệ nền tảng	17
2.1 Hệ thống quản lý tài liệu bảo mật	17
2.2 Kiến trúc microservices	17
2.3 Xác thực và phân quyền	18
2.3.1 Keycloak, OIDC và JWT	18
2.3.2 RBAC	18
2.3.3 ACL và deny-overrides	19
2.4 Phân loại tài liệu và kiểm soát nội dung	19
2.5 Audit, hash-chain và compliance evidence	19
2.6 DevSecOps	20
2.7 Các nhóm kiểm thử bảo mật trong pipeline	20
2.8 GitOps và Argo CD	20
3 Phân tích yêu cầu và thiết kế tổng thể	22
3.1 Tổng quan bài toán	22
3.2 Tác nhân hệ thống	23

3.3	Yêu cầu chức năng	23
3.3.1	Xác thực và phân quyền	23
3.3.2	Quản lý tài liệu	23
3.3.3	Workflow phê duyệt	24
3.3.4	Preview và download	24
3.3.5	Audit, security và evidence	24
3.3.6	Retention và access review	24
3.4	Yêu cầu phi chức năng	25
3.5	Mô hình threat cơ bản	25
3.6	Kiến trúc tổng thể	26
3.7	Luồng nghiệp vụ chính	28
3.8	Nguyên tắc thiết kế	28
4	Thiết kế dữ liệu, API và chính sách bảo mật	30
4.1	Tổng quan thiết kế dữ liệu	30
4.2	Các thực thể chính	32
4.2.1	documents	32
4.2.2	document_versions	32
4.2.3	document_acl	32
4.2.4	document_workflow_history	32
4.2.5	audit_events	32
4.3	Document lifecycle	33
4.4	Ma trận phân quyền	35
4.5	Chính sách Compliance Officer	35
4.6	Thiết kế preview/download authorization	36
4.7	Malware scanning và DLP	38
4.8	Audit ingest boundary	38
4.9	Luồng compliance evidence	38
4.10	API contract tổng quan	39
5	Triển khai ứng dụng DocVault	41
5.1	Cấu trúc monorepo	41
5.2	Frontend Next.js	41
5.2.1	Permission-aware UI	43
5.3	Gateway	43
5.4	Metadata-service	44
5.5	Document-service	44
5.6	Workflow-service	44
5.7	Audit-service	45
5.8	Notification-service	45

5.9	Hạ tầng local	45
5.10	Các màn hình minh họa	45
5.11	Luồng demo ứng dụng	47
6	Quy trình DevSecOps và hạ tầng triển khai	49
6.1	Mục tiêu của DevSecOps pipeline	49
6.2	Jenkins pipeline tổng thể	49
6.3	Pre-build Security	50
6.4	Build và scan image	51
6.5	Push image và GitOps update	51
6.6	Argo CD health check	51
6.7	Post-deploy smoke test	51
6.8	DAST bằng OWASP ZAP	52
6.9	Observability	52
6.10	Mapping stage, công cụ và bằng chứng	52
6.11	Trạng thái hiện tại của pipeline	53
7	Kiểm thử, đánh giá và bằng chứng	54
7.1	Chiến lược kiểm thử	54
7.2	Kiểm thử runtime	54
7.3	Kiểm thử frontend và UI evidence	55
7.4	Security evidence	56
7.5	Pipeline evidence	57
7.6	Đánh giá theo yêu cầu	57
7.7	Hạn chế hiện tại	58
7.8	Nhận xét chung	59
8	Kết luận và hướng phát triển	60
8.1	Kết quả đạt được	60
8.2	Mức độ đáp ứng mục tiêu	60
8.3	Hạn chế	61
8.4	Hướng phát triển	61
8.5	Kết luận chung	62
A	Hướng dẫn chạy hệ thống	63
A.1	Yêu cầu môi trường	63
A.2	Cài đặt phụ thuộc	63
A.3	Khởi động hạ tầng local	63
A.4	Chạy migration	63
A.5	Khởi động backend	64

A.6	Khởi động frontend	64
A.7	Tài khoản demo	64
A.8	Kiểm thử nhanh	65
B	Tóm tắt API contract	66
B.1	Nguyên tắc chung	66
B.2	Nhóm endpoint chính	66
B.3	Compliance Officer rule	67
B.4	Grant token	67
C	Pipeline evidence	68
C.1	Tham số Jenkins khuyến nghị	68
C.2	Bảng chứng cần chụp	68
C.3	Diễn giải khi bảo vệ	69
D	Danh sách hình ảnh minh họa	70
D.1	Ảnh UI đã sử dụng	70
D.2	Ảnh UI nên bổ sung	70
D.3	Ảnh cần bổ sung trước khi nộp	70

Danh sách hình vẽ

3.1	Kiến trúc tổng thể ứng dụng DocVault	27
3.2	Luồng nghiệp vụ demo chính theo vai trò trong DocVault	28
4.1	ERD và ranh giới sở hữu dữ liệu trong DocVault	31
4.2	Vòng đời trạng thái tài liệu trong DocVault	34
4.3	Sequence cấp quyền download và stream file	37
4.4	Luồng tổng hợp compliance evidence trong DocVault	39
5.1	Màn hình dashboard của DocVault	42
5.2	Màn hình danh sách tài liệu	43
5.3	Màn hình rà soát quyền truy cập	46
5.4	Màn hình phê duyệt tài liệu	46
5.5	Màn hình audit log	47
5.6	Màn hình security posture	47
7.1	Bảng chứng giao diện dashboard command center	56
7.2	Bảng chứng giao diện hàng đợi phê duyệt và trạng thái SLA	56

Danh sách bảng

1.1	Phạm vi và trạng thái thực hiện của đề tài	15
2.1	Các nhóm kiểm thử bảo mật trong DevSecOps pipeline	20
3.1	Các tác nhân chính trong DocVault	23
3.2	Yêu cầu phi chức năng	25
4.1	Ma trận quyền tổng quan theo vai trò	35
4.2	Nhóm API chính của DocVault	40
6.1	Mapping pipeline stage với công cụ và bằng chứng	52
7.1	Tóm tắt bằng chứng kiểm thử runtime	55
7.2	Mapping yêu cầu với hiện thực và bằng chứng	58
B.1	Tóm tắt endpoint chính	66
C.1	Danh sách pipeline evidence cần đưa vào khi nộp	68

Danh mục từ viết tắt

Từ viết tắt	Tiếng Anh	Diễn giải trong báo cáo
DMS	Document Management System	Hệ thống quản lý tài liệu.
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò.
ACL	Access Control List	Danh sách quyền truy cập theo người dùng, vai trò, nhóm hoặc tất cả.
OIDC	OpenID Connect	Lớp định danh dùng trên OAuth 2.0, được Keycloak sử dụng cho đăng nhập.
JWT	JSON Web Token	Token mang thông tin định danh và vai trò của người dùng.
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp, kiểm thử, đóng gói và triển khai tự động.
DevSecOps	Development, Security, Operations	Tích hợp bảo mật vào quy trình phát triển và vận hành.
SAST	Static Application Security Testing	Kiểm thử bảo mật bằng phân tích mã nguồn tĩnh.
SCA	Software Composition Analysis	Phân tích thành phần phụ thuộc và lỗ hổng thư viện.
DAST	Dynamic Application Security Testing	Kiểm thử bảo mật động trên ứng dụng đang chạy.
IaC	Infrastructure as Code	Mô tả hạ tầng bằng mã, ví dụ Terraform/Kubernetes manifest.
SBOM	Software Bill of Materials	Danh mục thành phần phần mềm và phụ thuộc.
DLP	Data Loss Prevention	Cơ chế phát hiện và giảm rủi ro rò rỉ dữ liệu nhạy cảm.
EKS	Elastic Kubernetes Service	Dịch vụ Kubernetes được quản lý trên AWS.

GitOps

Git Operations

Mô hình dùng Git làm nguồn sự thật
cho trạng thái triển khai.

Chương 1

Tổng quan đề tài

1.1 Bối cảnh

Trong các tổ chức hiện đại, tài liệu không chỉ là dữ liệu văn phòng thông thường mà còn là tài sản thông tin có giá trị. Một tài liệu có thể chứa hợp đồng, báo cáo tài chính, hồ sơ nhân sự, thông tin khách hàng, biên bản phê duyệt hoặc dữ liệu có yêu cầu bảo mật cao. Khi số lượng tài liệu tăng, nhu cầu quản lý vòng đời tài liệu, phân quyền truy cập, kiểm soát tải xuống và truy vết hoạt động trở thành một yêu cầu thiết yếu.

Các hệ thống quản lý tài liệu truyền thống thường tập trung vào việc lưu trữ và tìm kiếm. Tuy nhiên, trong bối cảnh an toàn thông tin, hệ thống quản lý tài liệu cần trả lời thêm nhiều câu hỏi: ai được xem metadata, ai được tải file, tài liệu nào được xem là nhạy cảm, ai đã phê duyệt tài liệu, thao tác nào bị từ chối, file tải lên có chứa mã độc hay dữ liệu nhạy cảm không, và bằng chứng nào có thể cung cấp cho quá trình kiểm tra tuân thủ. Những yêu cầu này khiến bài toán quản lý tài liệu trở thành bài toán kết hợp giữa nghiệp vụ, bảo mật và vận hành.

Đồng thời, quy trình phát triển phần mềm hiện đại không còn xem bảo mật là một bước kiểm tra cuối cùng. DevSecOps khuyến khích đưa các kiểm soát bảo mật vào sớm và xuyên suốt vòng đời phát triển: từ mã nguồn, thư viện phụ thuộc, cấu hình hạ tầng, Docker image cho đến ứng dụng sau khi triển khai. Theo khuyến nghị của NIST SSDF và các hướng dẫn DevSecOps của OWASP, việc tự động hóa kiểm thử và thu thập bằng chứng bảo mật giúp giảm rủi ro và tăng tính lặp lại của quy trình phát hành [1], [2].

Từ bối cảnh trên, đề tài DocVault được xây dựng với mục tiêu triển khai một hệ thống quản lý tài liệu bảo mật, đồng thời thiết kế quy trình DevSecOps phục vụ kiểm thử, đóng gói và triển khai hệ thống.

1.2 Vấn đề cần giải quyết

Đề tài tập trung vào hai nhóm vấn đề chính. Nhóm thứ nhất là vấn đề ứng dụng: xây dựng hệ thống quản lý tài liệu có phân quyền rõ ràng, có quy trình phê duyệt, có lưu trữ file an toàn và có audit trail đủ để phục vụ truy vết. Nhóm thứ hai là vấn đề quy trình: đưa các kiểm soát bảo mật vào pipeline để mỗi lần thay đổi mã nguồn đều có thể được kiểm tra bằng các bước tự động.

Các yêu cầu tiêu biểu có thể tóm tắt như sau:

- Người dùng thuộc các vai trò khác nhau phải nhìn thấy và thao tác trên tài liệu theo đúng phạm vi quyền hạn.
- Tài liệu hỗ trợ bốn mức phân loại: public, internal, confidential và secret.
- Luồng nghiệp vụ phải hỗ trợ tạo bản nháp, tải file, gửi duyệt, phê duyệt, từ chối và lưu trữ.
- File nhạy cảm không nên được cấp URL tải trực tiếp nếu cần đi qua đường stream có kiểm soát.
- Compliance Officer được xem metadata, audit và bằng chứng tuân thủ, nhưng không được truy cập nội dung file.
- Các thao tác quan trọng phải được audit và có khả năng kiểm tra tính toàn vẹn.
- Pipeline phải có các bước kiểm thử và quét bảo mật trước, trong và sau khi build/deploy.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng và trình bày một hệ thống quản lý tài liệu bảo mật tên DocVault theo hướng microservices, có tích hợp các kiểm soát bảo mật ở cả runtime và quy trình DevSecOps.

Các mục tiêu cụ thể gồm:

1. Thiết kế kiến trúc tổng thể cho hệ thống quản lý tài liệu bảo mật, gồm frontend, API Gateway, các microservice nghiệp vụ và các thành phần hạ tầng.
2. Xây dựng các chức năng chính của hệ thống: đăng nhập, quản lý tài liệu, version file, workflow phê duyệt, phân quyền, preview/download, audit, security dashboard, evidence và retention.
3. Thiết kế mô hình phân quyền dựa trên RBAC, ACL, trạng thái tài liệu, phân loại bảo mật và quy tắc deny-overrides.
4. Xây dựng cơ chế audit hash-chain và các bằng chứng phục vụ compliance.
5. Tích hợp các kiểm soát bảo mật như malware scanning, DLP, grant token ngắn hạn và kiểm soát truy cập nội dung file.

6. Xây dựng pipeline DevSecOps với Jenkins, SonarQube, Dependency Check, Trivy, Checkov, Terraform validation, Docker build/scan, GitOps, Argo CD, smoke test và OWASP ZAP.
7. Kiểm thử và đánh giá hệ thống dựa trên các kịch bản runtime, security evidence và pipeline evidence.

1.4 Phạm vi thực hiện

Phạm vi đề tài được chia thành hai phần. Phần ứng dụng DocVault là trọng tâm hiện tại và đã được triển khai gần hoàn thiện. Phần này bao gồm giao diện web, backend microservices, các luồng nghiệp vụ, kiểm soát truy cập, audit, DLP, malware scanning, retention và evidence center.

Phần DevSecOps pipeline đang trong quá trình hoàn thiện. Pipeline đã có cấu trúc và các stage quan trọng như SAST, SCA, Trivy, Checkov, Terraform validation, build/scan image, push image, GitOps update, Argo CD health check, smoke test và DAST bằng OWASP ZAP. Tuy nhiên, trong phạm vi báo cáo, phần pipeline được trình bày trung thực theo trạng thái hiện có: đã có thiết kế và bằng chứng chạy, nhưng vẫn còn các hướng cải tiến về production hardening, artifact signing, SBOM, policy-as-code và độ đầy đủ của bằng chứng triển khai.

Bảng 1.1: Phạm vi và trạng thái thực hiện của đề tài

Nhóm nội dung	Mô tả	Trạng thái
Ứng dụng web	Frontend Next.js, các màn hình dashboard, documents, approvals, audit, security, evidence, retention.	Đã triển khai
Backend microservices	Gateway, metadata-service, document-service, workflow-service, audit-service, notification-service.	Đã triển khai
Bảo mật runtime	RBAC, ACL, grant token, compliance boundary, DLP, malware scanning, audit hash-chain.	Đã triển khai
Local infrastructure	Postgres, MongoDB, MinIO, Keycloak qua Docker Compose.	Đã triển khai
DevSecOps pipeline	Jenkins pipeline với các stage kiểm thử, scan, build, GitOps, Argo CD, smoke test, DAST.	Đang hoàn thiện
Production hardening	SBOM, signing, policy-as-code, secret rotation/KMS, advanced observability.	Cần bổ sung

1.5 Đóng góp của đề tài

Đề tài có bốn đóng góp chính. Thứ nhất, đề tài xây dựng một hệ thống quản lý tài liệu có đầy đủ luồng nghiệp vụ từ tạo tài liệu đến phê duyệt và lưu trữ. Thứ hai, hệ thống thể hiện rõ cách kết hợp các mô hình kiểm soát truy cập ở nhiều tầng: role, ACL, classification, status và ownership. Thứ ba, đề tài đưa các yêu cầu compliance vào thiết kế thông qua audit hash-chain, retention evidence và evidence packet. Thứ tư, đề tài xây dựng pipeline DevSecOps nhằm tự động hóa kiểm thử và quét bảo mật, qua đó tạo nền tảng cho quá trình phát hành an toàn hơn.

1.6 Cấu trúc báo cáo

Báo cáo gồm tám chương chính. Chương 1 trình bày bối cảnh, mục tiêu và phạm vi đề tài. Chương 2 trình bày cơ sở lý thuyết và công nghệ nền tảng. Chương 3 phân tích yêu cầu và thiết kế tổng thể. Chương 4 mô tả thiết kế dữ liệu, API và chính sách bảo mật. Chương 5 trình bày quá trình triển khai ứng dụng DocVault. Chương 6 trình bày quy trình DevSecOps và hạ tầng triển khai. Chương 7 mô tả kiểm thử, đánh giá và bằng

chúng. Chương 8 tổng kết kết quả đạt được, hạn chế và hướng phát triển.

Chương 2

Cơ sở lý thuyết và công nghệ nền tảng

2.1 Hệ thống quản lý tài liệu bảo mật

Hệ thống quản lý tài liệu (Document Management System - DMS) là hệ thống hỗ trợ lưu trữ, tổ chức, tìm kiếm, chia sẻ và kiểm soát vòng đời tài liệu. Trong môi trường doanh nghiệp, DMS không chỉ quản lý file mà còn quản lý metadata, phiên bản, người sở hữu, trạng thái phê duyệt và lịch sử thao tác.

Đối với tài liệu có tính nhạy cảm, DMS cần bổ sung các yêu cầu bảo mật. Một tài liệu không nên chỉ được bảo vệ bằng vị trí lưu trữ, mà cần được bảo vệ bằng chính sách truy cập nhất quán ở backend. Điều này đặc biệt quan trọng với các thao tác như preview, stream, presign download, export evidence hoặc truy vấn audit. Nếu hệ thống chỉ ẩn nút thao tác ở frontend nhưng backend vẫn cho phép gọi API, chính sách bảo mật không còn đáng tin cậy.

DocVault được thiết kế theo quan điểm backend là nơi thực thi chính sách cuối cùng. Frontend có thể hiển thị hoặc vô hiệu hóa thao tác để cải thiện trải nghiệm người dùng, nhưng quyết định cho phép hoặc từ chối vẫn thuộc về metadata-service, document-service, gateway và audit-service.

2.2 Kiến trúc microservices

Microservices là cách tổ chức hệ thống thành nhiều service nhỏ, mỗi service sở hữu một năng lực nghiệp vụ rõ ràng và có thể phát triển, kiểm thử, triển khai độc lập ở mức tương đối. Trong DocVault, kiến trúc này được dùng để tách các bounded context:

- Gateway chịu trách nhiệm xác thực, routing và truyền ngữ cảnh.
- Metadata-service sở hữu metadata, ACL, trạng thái và policy.
- Document-service sở hữu upload/download file và tích hợp object storage.

- Workflow-service sở hữu state machine phê duyệt.
- Audit-service sở hữu audit trail và hash-chain.
- Notification-service nhận thông báo nghiệp vụ trong môi trường demo/dev.

Cách tách này giúp giảm sự phụ thuộc giữa lưu trữ file và chính sách truy cập. File thật được lưu ở MinIO/S3, trong khi metadata và quyền truy cập nằm ở service khác. Khi người dùng muốn tải file, document-service không tự ý quyết định quyền mà phải dựa vào policy/grant token do metadata-service cấp hoặc xác minh lại.

2.3 Xác thực và phân quyền

2.3.1 Keycloak, OIDC và JWT

Keycloak là hệ thống quản lý định danh và truy cập mã nguồn mở, hỗ trợ các chuẩn phổ biến như OAuth 2.0 và OpenID Connect [3]. Trong DocVault, Keycloak được sử dụng để quản lý người dùng, vai trò, nhóm và phát hành token. Sau khi đăng nhập, người dùng nhận JWT chứa thông tin định danh và các claim cần thiết như user id, username, roles và groups.

Gateway và các service phía sau sử dụng JWT để xác định actor của request. Gateway là entry point chính, xác thực token và truyền các header ngữ cảnh như **x-user-id**, **x-roles**, **x-groups** xuống các service nội bộ. Cách này giúp các service nghiệp vụ không cần phụ thuộc trực tiếp vào phiên đăng nhập frontend, nhưng vẫn đủ thông tin để thực thi chính sách.

2.3.2 RBAC

RBAC là mô hình phân quyền theo vai trò. Thay vì gán quyền trực tiếp cho từng người dùng, hệ thống gán quyền cho vai trò, sau đó người dùng được gán một hoặc nhiều vai trò. Trong DocVault, các vai trò chính gồm:

- **viewer**: đọc tài liệu đã xuất bản nếu policy cho phép.
- **editor**: tạo tài liệu, chỉnh sửa metadata, upload file và gửi duyệt.
- **approver**: xem và phê duyệt hoặc từ chối tài liệu đang chờ duyệt.
- **compliance_officer**: xem audit, metadata và evidence trong phạm vi compliance, nhưng không được truy cập nội dung file.
- **admin**: quản trị hệ thống và có quyền rộng nhất.

RBAC phù hợp để kiểm soát quyền ở mức route hoặc chức năng, nhưng chưa đủ để mô hình hóa mọi trường hợp truy cập theo từng tài liệu. Vì vậy, DocVault bổ sung ACL.

2.3.3 ACL và deny-overrides

ACL cho phép gán rule truy cập ở mức tài liệu. Một rule có thể áp dụng cho user, role, group hoặc tất cả người dùng. Mỗi rule có permission như READ, DOWNLOAD, WRITE, APPROVE và effect là ALLOW hoặc DENY.

DocVault sử dụng chiến lược deny-overrides: nếu có rule DENY phù hợp, kết quả cuối cùng là từ chối, ngay cả khi có rule ALLOW khác. Đây là chiến lược an toàn hơn cho dữ liệu nhạy cảm, vì một lệnh cấm rõ ràng không bị ghi đè bởi quyền rộng hơn.

2.4 Phân loại tài liệu và kiểm soát nội dung

DocVault sử dụng bốn mức phân loại tài liệu:

- PUBLIC: tài liệu công khai trong phạm vi người dùng đã xác thực.
- INTERNAL: tài liệu nội bộ.
- CONFIDENTIAL: tài liệu mật, cần kiểm soát preview/download nghiêm ngặt hơn.
- SECRET: tài liệu rất nhạy cảm, chỉ một số vai trò/đối tượng được phép truy cập.

Mức phân loại không chỉ là metadata hiển thị mà còn ảnh hưởng đến chính sách truy cập. Ví dụ, tài liệu CONFIDENTIAL hoặc SECRET có thể không trả về presigned URL trực tiếp, mà yêu cầu đi qua endpoint stream để document-service có thể áp dụng kiểm soát bổ sung như watermark posture và tái xác thực grant.

2.5 Audit, hash-chain và compliance evidence

Audit log ghi lại các sự kiện quan trọng trong hệ thống: tạo tài liệu, upload file, submit, approve, reject, download authorized/denied, DLP detected, malware blocked, retention archive và các thao tác compliance. Một audit log hữu ích không chỉ cần có dữ liệu, mà còn cần có khả năng kiểm tra tính toàn vẹn.

DocVault sử dụng hash-chain cho audit events. Mỗi event có hash phụ thuộc vào nội dung event hiện tại và hash của event trước đó. Nếu một event cũ bị sửa trực tiếp trong storage, chuỗi hash phía sau sẽ không còn khớp và verify-chain có thể phát hiện bất thường. Cơ chế này không biến audit log thành lưu trữ bất biến tuyệt đối, nhưng tạo tính tamper-evident phù hợp với phạm vi đồ án.

Bên cạnh audit query, hệ thống còn cung cấp evidence packet. Evidence packet gồm metadata, version checksum, workflow history, retention evidence, trạng thái verify-chain và các audit event liên quan đến một tài liệu. Điểm quan trọng là evidence packet không chứa nội dung file, presigned URL, grant token hoặc object key nhạy cảm.

2.6 DevSecOps

DevSecOps là cách tiếp cận tích hợp bảo mật vào quy trình phát triển và vận hành. Thay vì kiểm tra bảo mật ở cuối dự án, DevSecOps đưa các kiểm soát vào nhiều điểm trong pipeline: kiểm tra mã nguồn, kiểm tra thư viện phụ thuộc, kiểm tra hạ tầng, kiểm tra Docker image, kiểm tra ứng dụng sau khi triển khai và thu thập artifact làm bằng chứng.

Trong DocVault, DevSecOps được thể hiện qua Jenkins pipeline. Pipeline không chỉ chạy test mà còn chạy SAST, SCA, Trivy filesystem scan, Trivy image scan, Checkov IaC scan, Terraform validate, smoke test và DAST bằng OWASP ZAP. Những bước này phù hợp với tư tưởng “shift-left security” được đề cập trong các tài liệu DevSecOps [1], [2].

2.7 Các nhóm kiểm thử bảo mật trong pipeline

Bảng 2.1: Các nhóm kiểm thử bảo mật trong DevSecOps pipeline

Nhóm	Mục tiêu	Ví dụ áp dụng trong DocVault	Công cụ
SAST	Phân tích mã nguồn tĩnh để phát hiện lỗi chất lượng và bảo mật.	Kiểm tra backend/frontend TypeScript, quality gate và code smell.	SonarQube
SCA	Phân tích thư viện phụ thuộc và lỗ hổng đã biết.	Quét dependency của monorepo Node.js.	OWASP Dependency Check
FS scan	Quét filesystem repository.	Phát hiện lỗ hổng trong file hệ thống, lockfile, cấu hình.	Trivy
Image scan	Quét Docker image sau build.	Phát hiện CVE trong image backend/frontend.	Trivy
IaC scan	Quét cấu hình hạ tầng.	Kiểm tra Terraform/Kubernetes manifest.	Checkov
DAST	Kiểm thử ứng dụng đang chạy.	Baseline scan web app sau deploy.	OWASP ZAP

2.8 GitOps và Argo CD

GitOps là mô hình trong đó Git đóng vai trò nguồn sự thật cho trạng thái triển khai. Khi pipeline build image mới, thay vì trực tiếp chỉnh cluster bằng lệnh thủ công, pipeline cập

nhật Helm values hoặc manifest trong một branch GitOps. Argo CD theo dõi branch này và đồng bộ trạng thái mong muốn vào Kubernetes cluster [4].

Trong DocVault, pipeline cập nhật các file values tương ứng với từng service. Sau đó Argo CD thực hiện sync các application như gateway, metadata-service, document-service, workflow-service, audit-service, notification-service và web. Cách tiếp cận này giúp tách trách nhiệm giữa build pipeline và deployment controller, đồng thời tạo lịch sử thay đổi triển khai trong Git.

Chương 3

Phân tích yêu cầu và thiết kế tổng thể

3.1 Tổng quan bài toán

DocVault hướng đến bài toán quản lý tài liệu trong môi trường có nhiều vai trò và nhiều mức độ nhạy cảm dữ liệu. Một tài liệu đi qua nhiều trạng thái: bản nháp, chờ duyệt, đã xuất bản và đã lưu trữ. Trong quá trình đó, mỗi vai trò chỉ được thực hiện những thao tác phù hợp. Editor có thể tạo và gửi duyệt tài liệu; Approver có thể phê duyệt hoặc từ chối; Viewer chỉ được truy cập tài liệu đã xuất bản nếu policy cho phép; Compliance Officer có thể xem audit và evidence nhưng không được truy cập nội dung file; Admin có quyền quản trị rộng hơn.

Điểm cốt lõi của bài toán không chỉ là lưu file, mà là kiểm soát file theo ngữ cảnh. Cùng một file có thể được phép preview với người này nhưng bị từ chối với người khác; có thể được cấp presigned URL khi là tài liệu public nhưng phải stream qua backend khi là tài liệu confidential; có thể cho phép xem metadata nhưng không cho tải nội dung. Vì vậy, hệ thống cần mô hình policy đủ rõ và được thực thi ở backend.

3.2 Tác nhân hệ thống

Bảng 3.1: Các tác nhân chính trong DocVault

Tác nhân	Vai trò trong hệ thống
Viewer	Người dùng đọc tài liệu đã được xuất bản trong phạm vi policy cho phép; có thể preview/download tài liệu public hoặc tài liệu được cấp quyền.
Editor	Người tạo và quản lý tài liệu của mình; có thể tạo draft, chỉnh metadata, upload phiên bản, gửi duyệt và archive khi có quyền.
Approver	Người kiểm tra tài liệu đang ở trạng thái chờ duyệt; có thể approve hoặc reject theo quy trình.
Compliance Officer	Người kiểm tra tuân thủ; được xem audit, metadata/evidence được phép, nhưng bị chặn preview, stream, presign và download nội dung file.
Admin	Người quản trị hệ thống; có quyền rộng hơn trên tài liệu, thành viên, evidence và retention.

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo miền nghiệp vụ như sau:

3.3.1 Xác thực và phân quyền

Hệ thống cần hỗ trợ đăng nhập thông qua Keycloak, đọc thông tin người dùng hiện tại và phân quyền theo vai trò. Navigation, command palette và các thao tác trên UI cần thay đổi theo role, nhưng backend vẫn phải là nơi quyết định cuối cùng. Các request không có token, token hết hạn hoặc token không hợp lệ phải bị từ chối.

3.3.2 Quản lý tài liệu

Người dùng có quyền có thể tạo tài liệu mới với tiêu đề, mô tả, classification và tags. Sau khi tạo, tài liệu ở trạng thái **DRAFT**. Editor hoặc Admin có thể upload file ban đầu hoặc upload phiên bản mới. Hệ thống cần lưu checksum, kích thước, MIME type, tên file và người upload cho từng version.

3.3.3 Workflow phê duyệt

Tài liệu đi qua vòng đời chính DRAFT -> PENDING -> PUBLISHED -> ARCHIVED. Editor gửi tài liệu từ Draft sang Pending. Approver phê duyệt tài liệu Pending sang Published hoặc từ chối để quay về Draft. Tài liệu Published có thể được archive theo quyền. Mỗi transition cần ghi workflow history và audit event.

3.3.4 Preview và download

Người dùng có quyền có thể preview hoặc download tài liệu tùy trạng thái, classification, role, ownership và ACL. Với tài liệu nhạy cảm, hệ thống có thể không trả về URL tải trực tiếp mà trả về endpoint stream có kiểm soát. Compliance Officer luôn bị chặn truy cập nội dung file.

3.3.5 Audit, security và evidence

Hệ thống cần ghi audit event cho các hành động quan trọng, hỗ trợ truy vấn audit theo bộ lọc, kiểm tra hash-chain và hiển thị security summary. Compliance/Admin có thể xuất evidence packet liên quan đến tài liệu, recommendation hoặc compliance flow.

3.3.6 Retention và access review

Tài liệu đã xuất bản cần có retention evidence dựa trên classification. Admin có thể chạy retention job demo để auto-archive các tài liệu đến hạn. Compliance/Admin có thể xem access review cho tài liệu nhạy cảm, broad ACL grant hoặc quyền đã cũ.

3.4 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng

Nhóm yêu cầu	Mô tả
Bảo mật	Backend phải enforce policy; token và grant phải được xác minh; nội dung file không được lộ qua evidence hoặc audit.
Truy vết	Các thao tác quan trọng cần tạo audit event; audit chain cần có khả năng verify.
Tách biệt trách nhiệm	Mỗi service sở hữu một bounded context rõ ràng; document-service không tự quyết định policy download.
Khả năng kiểm thử	Có unit test, E2E runtime, frontend test/lint/build và pipeline scan.
Khả năng triển khai	Hỗ trợ local Docker Compose, Docker image, Kubernetes/Helm và GitOps.
Khả năng mở rộng	Thiết kế cho phép bổ sung queue, email notification, SBOM, signing, KMS và policy-as-code trong tương lai.

3.5 Mô hình threat cơ bản

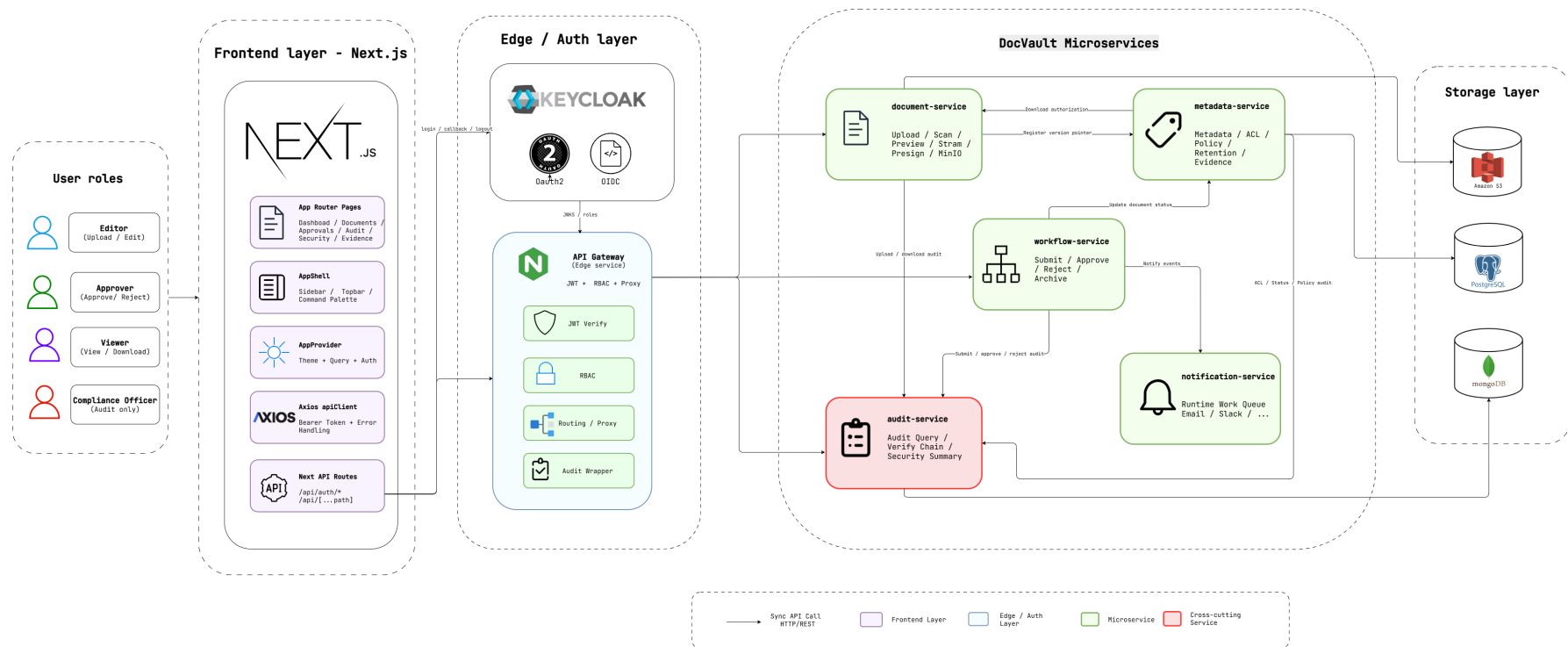
Để định hướng thiết kế bảo mật, hệ thống xem xét các nhóm rủi ro sau:

- **Truy cập trái phép metadata:** người dùng đoán document id và gọi API chi tiết, workflow history, comments hoặc ACL.
- **Tải nội dung file trái phép:** người dùng gọi trực tiếp presign, stream hoặc download endpoint mà không có quyền.
- **Lạm dụng Compliance Officer:** người có vai trò compliance được xem nhiều metadata nhưng không được dùng vai trò này để lấy file.
- **Upload file độc hại:** file chứa EICAR/malware không được lưu vào MinIO và không tạo version metadata.
- **Rò rỉ dữ liệu nhạy cảm:** file public/internal chứa email, số điện thoại, từ khóa confidential/secret cần được phát hiện và nâng classification.
- **Downgrade classification không hợp lệ:** tài liệu đã bị DLP phát hiện không được hạ xuống public/internal nếu không có quyền và lý do hợp lệ.
- **Giả mạo audit:** người dùng thường không được append audit event trực tiếp; audit ingest cần service token.
- **Sửa audit log cũ:** hash-chain cần phát hiện khi audit event bị thay đổi trực tiếp trong storage.

3.6 Kiến trúc tổng thể

Kiến trúc DocVault gồm bốn lớp: frontend, gateway, microservices và hạ tầng dữ liệu. Frontend Next.js là giao diện người dùng. API Gateway là điểm vào duy nhất cho frontend, chịu trách nhiệm xác thực, role guard, proxy request và truyền ngữ cảnh. Các service phía sau xử lý nghiệp vụ riêng. Hạ tầng dữ liệu gồm Keycloak, PostgreSQL, MongoDB và MinIO/S3.

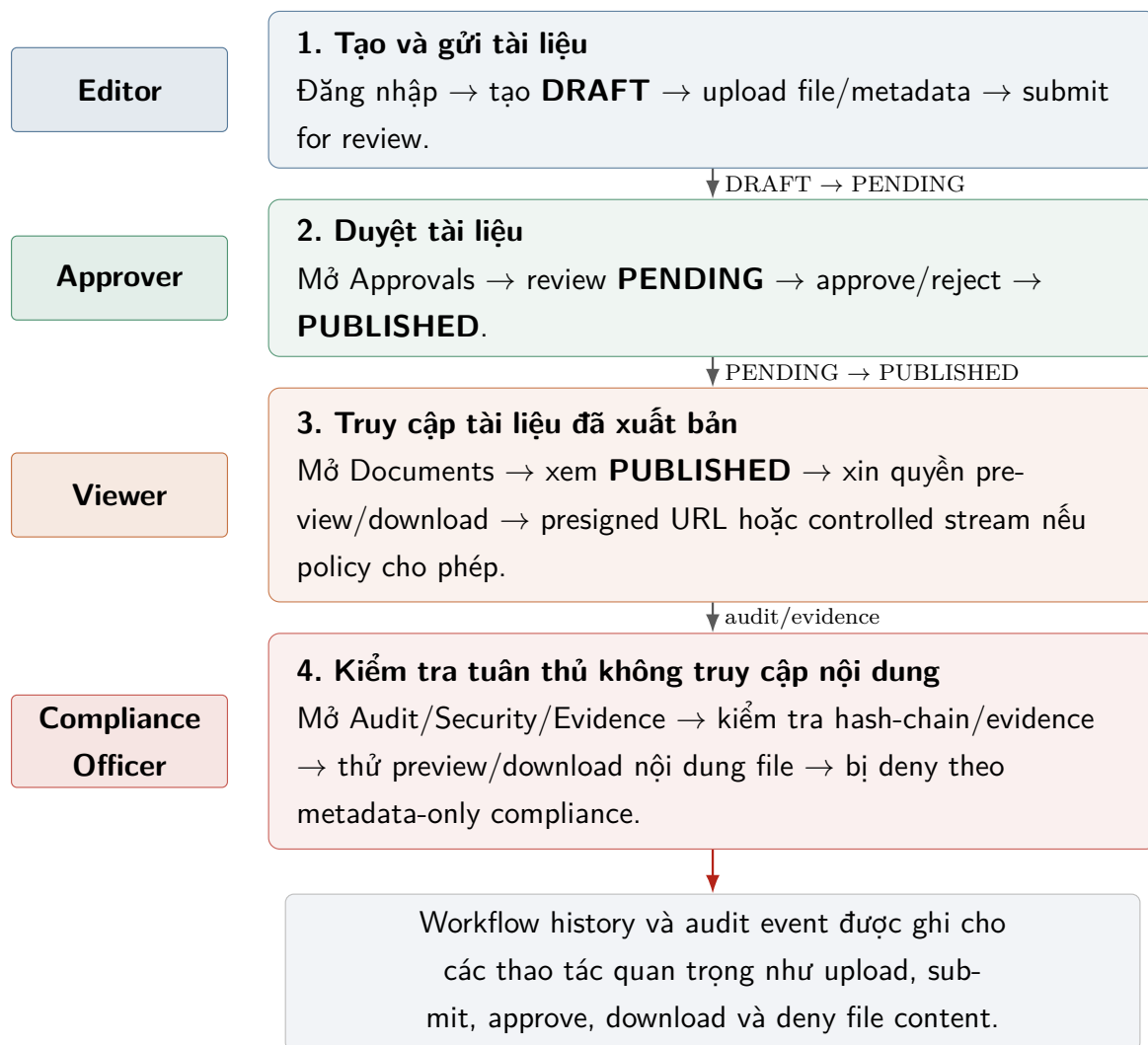
Hình 3.1 mô tả cách các lớp này liên kết với nhau. Điểm quan trọng của kiến trúc là frontend chỉ giao tiếp qua gateway, còn các microservice phía sau giữ ranh giới nghiệp vụ riêng: metadata-service sở hữu policy và metadata, document-service sở hữu blob file, workflow-service điều phối trạng thái, audit-service ghi nhận sự kiện và notification-service xử lý thông báo.



Hình 3.1: Kiến trúc tổng thể ứng dụng DocVault

3.7 Luồng nghiệp vụ chính

Luồng demo chính của hệ thống gồm bốn vai trò. Editor đăng nhập, tạo tài liệu, upload file và submit. Approver đăng nhập, kiểm tra tài liệu pending và approve. Viewer đăng nhập, xem tài liệu đã published và download nếu policy cho phép. Compliance Officer đăng nhập, xem audit/evidence nhưng bị chặn preview/download nội dung file.



Hình 3.2: Luồng nghiệp vụ demo chính theo vai trò trong DocVault

3.8 Nguyên tắc thiết kế

Thiết kế DocVault tuân theo các nguyên tắc sau:

1. **Backend authoritative**: frontend chỉ hỗ trợ UX; backend luôn là nơi thực thi policy.
2. **Tách metadata và blob**: metadata nằm ở PostgreSQL, file nằm ở MinIO/S3, version metadata chỉ giữ object key và checksum.

3. **Deny wins:** ACL deny có độ ưu tiên cao hơn allow.
4. **Metadata-only compliance:** compliance có thể kiểm tra hệ thống mà không cần truy cập nội dung file.
5. **Evidence without leakage:** evidence không chứa file content, presigned URL, grant token hoặc object key nhạy cảm.
6. **Security as pipeline gates:** các bước scan/test cần chạy tự động trong pipeline thay vì phụ thuộc thao tác thủ công.

Chương 4

Thiết kế dữ liệu, API và chính sách bảo mật

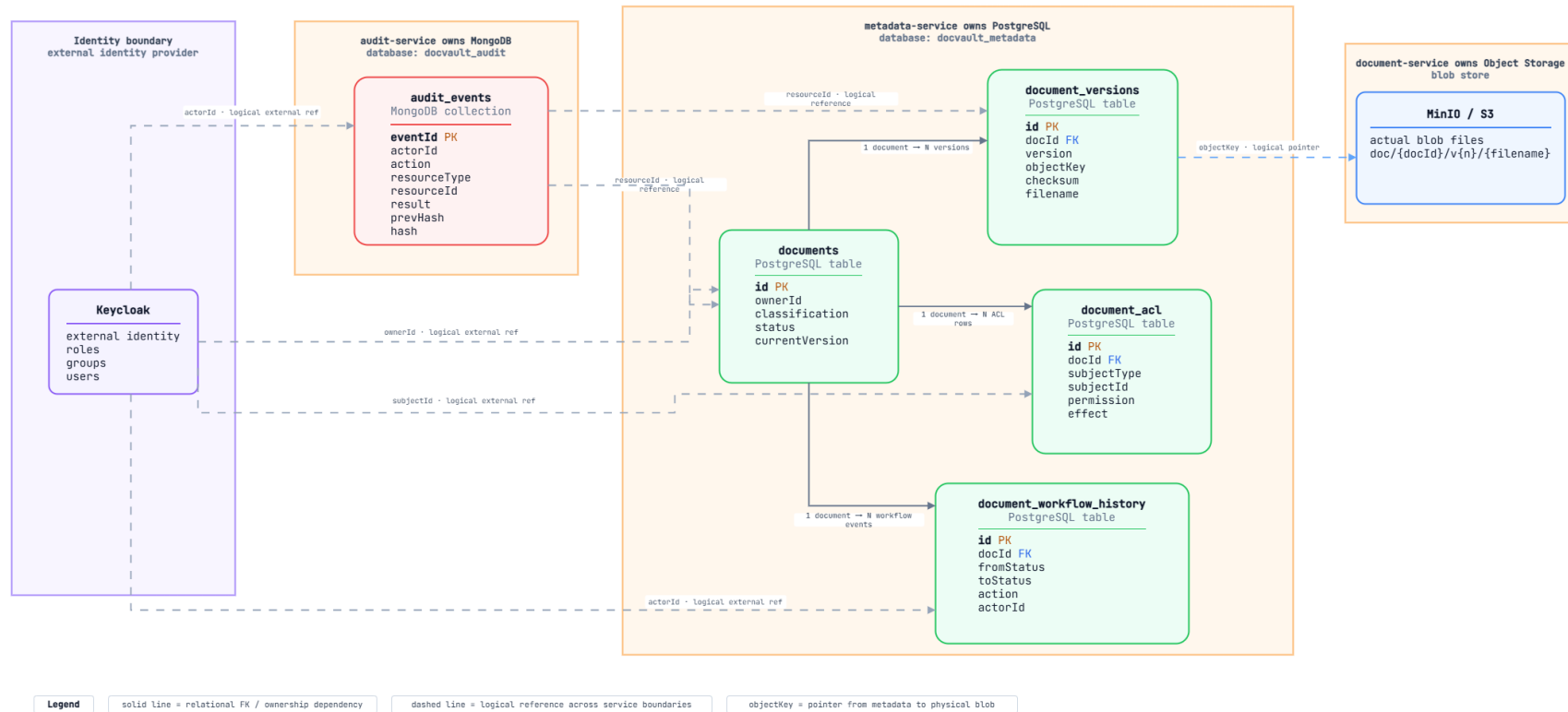
4.1 Tổng quan thiết kế dữ liệu

DocVault tách dữ liệu thành ba nhóm chính. Metadata tài liệu thuộc metadata-service và lưu trong PostgreSQL. Nội dung file thuộc document-service và lưu trong MinIO/S3. Audit event thuộc audit-service và lưu trong MongoDB. Việc tách này giúp mỗi service sở hữu dữ liệu của mình, tránh việc service khác truy vấn trực tiếp vào database không thuộc bounded context của nó.

Tài liệu nội bộ docs/ERD.md mô tả rõ nguyên tắc này: metadata database lưu documents, document versions, ACL và workflow history; audit database lưu audit events; object storage lưu blob file. Cách thiết kế này giúp hệ thống vừa có thể truy vấn metadata nhanh, vừa có thể lưu file dung lượng lớn trong object storage, đồng thời giữ audit log độc lập với nghiệp vụ tài liệu.

DocVault Data Ownership ERD

Service-owned data boundaries, logical external references, FK relationships, and object storage pointer



Hình 4.1: ERD và ranh giới sở hữu dữ liệu trong DocVault

4.2 Các thực thể chính

4.2.1 documents

Bảng **documents** là nguồn sự thật cho tài liệu logic. Mỗi record đại diện cho một tài liệu, gồm tiêu đề, mô tả, owner, classification, tags, status, current version và các mốc thời gian như `createdAt`, `updatedAt`, `publishedAt`, `archivedAt`.

Trạng thái tài liệu gồm **DRAFT**, **PENDING**, **PUBLISHED** và **ARCHIVED**. Classification gồm **PUBLIC**, **INTERNAL**, **CONFIDENTIAL** và **SECRET**. Hai trường này là nền tảng cho phần lớn quyết định policy.

4.2.2 document_versions

Bảng **document_versions** lưu thông tin từng phiên bản file. Mỗi version có `docId`, `version number`, `object key`, `checksum`, `size`, `filename`, `contentType` và `createdBy`. File thật không nằm trong PostgreSQL; database chỉ giữ pointer đến object storage.

Checksum SHA-256 hỗ trợ kiểm tra tính nhất quán và làm evidence. Khi export evidence packet, hệ thống có thể chứng minh tài liệu có version cụ thể và checksum cụ thể mà không cần đưa nội dung file vào packet.

4.2.3 document_acl

Bảng **document_acl** lưu các rule truy cập theo tài liệu. Rule có `subjectType`, `subjectId`, `permission` và `effect`. `SubjectType` hỗ trợ **USER**, **ROLE**, **GROUP** và **ALL**. `Permission` hỗ trợ **READ**, **DOWNLOAD**, **WRITE** và **APPROVE**. `Effect` là **ALLOW** hoặc **DENY**.

ACL không thay thế RBAC mà bổ sung cho RBAC. RBAC xác định năng lực chung theo vai trò, trong khi ACL xác định ngoại lệ hoặc quyền cụ thể theo tài liệu. Khi có rule deny phù hợp, hệ thống ưu tiên từ chối.

4.2.4 document_workflow_history

Bảng **document_workflow_history** lưu lịch sử chuyển trạng thái nghiệp vụ. Mỗi record có `docId`, `fromStatus`, `toStatus`, `action`, `actorId`, `reason` và `createdAt`. Bảng này phục vụ truy vấn nhanh lịch sử phê duyệt trong màn hình chi tiết tài liệu, tách khỏi audit-service vốn ghi nhiều loại sự kiện hệ thống hơn.

4.2.5 audit_events

Audit event ghi lại hành động trong hệ thống. Mỗi event chứa `actor`, `action`, `resource type`, `resource id`, `result`, `reason`, `metadata` và `hash`. Audit-service áp dụng hash-chain để

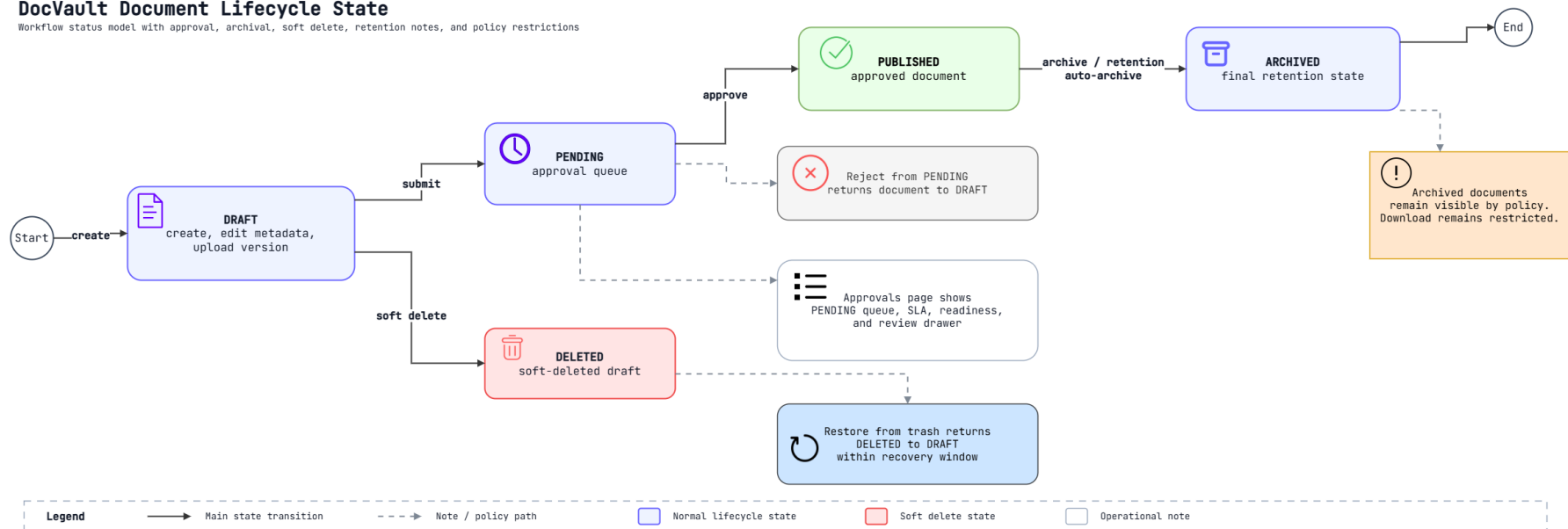
phát hiện sửa đổi ngoài luồng. Vì audit là bounded context riêng, audit event không dùng foreign key cứng đến documents.

4.3 Document lifecycle

Luồng trạng thái tài liệu trong DocVault được giữ đủ gọn để phù hợp với phạm vi MVP nhưng vẫn bao phủ demo doanh nghiệp. Các transition chính gồm tạo draft, gửi duyệt, phê duyệt, từ chối và lưu trữ; ngoài ra hệ thống còn mô phỏng soft delete và restore cho tài liệu nháp.

DocVault Document Lifecycle State

Workflow status model with approval, archival, soft delete, retention notes, and policy restrictions



Hình 4.2: Vòng đời trạng thái tài liệu trong DocVault

Khi Approver approve tài liệu, hệ thống set `publishedAt`. Khi archive, hệ thống set `archivedAt`. Mỗi transition được ghi vào workflow history và audit event. Điều này giúp trang chi tiết tài liệu hiển thị timeline nghiệp vụ, đồng thời compliance có thể đối chiếu với audit log.

4.4 Ma trận phân quyền

Policy của DocVault không chỉ dựa trên role. Kết quả truy cập được xác định bởi nhiều yếu tố: role, owner, ACL, classification, status, version existence và trạng thái deleted/archived. Tuy nhiên, ở mức tổng quan, ma trận dưới đây thể hiện ý nghĩa các vai trò chính.

Bảng 4.1: Ma trận quyền tổng quan theo vai trò

Vai trò	Quyền chính
Viewer	Xem danh sách/metadata và preview/download tài liệu đã published trong phạm vi policy.
Editor	Tạo tài liệu, upload version, chỉnh metadata, submit, archive tài liệu thuộc quyền sở hữu hoặc được cấp quyền.
Approver	Xem hàng đợi duyệt, kiểm tra tài liệu pending, approve hoặc reject.
Compliance Officer	Xem audit, security summary, retention, evidence và metadata được phép; luôn bị chặn nội dung file.
Admin	Quản trị tài liệu, thành viên, evidence, retention và các thao tác nhạy cảm.

4.5 Chính sách Compliance Officer

Một quyết định thiết kế quan trọng của DocVault là tách rõ quyền compliance và quyền nội dung file. Compliance Officer cần xem metadata và audit để đánh giá tuân thủ, nhưng không nên có quyền tải file chỉ vì họ có quyền audit. Điều này giảm rủi ro lạm dụng vai trò compliance để truy cập tài liệu nhạy cảm.

Trong DocVault, Compliance Officer:

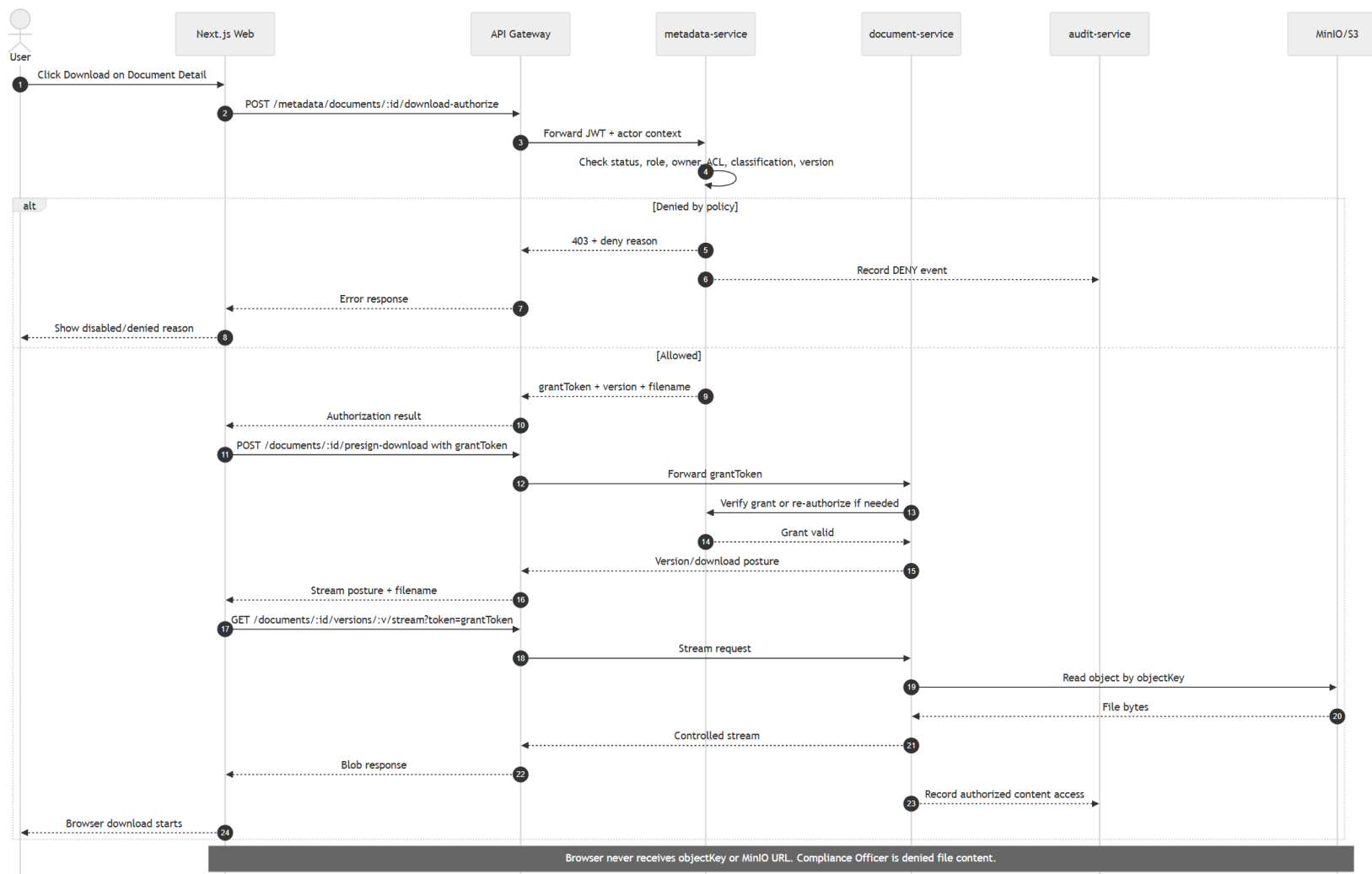
- được truy vấn audit events;
- được verify audit hash-chain;
- được xem security summary;
- được xem retention evidence;
- được export evidence packet;

- không được preview, stream, presign hoặc download nội dung file.

Quy tắc này được enforce ở nhiều tầng: frontend permission helper để hiển thị lý do deny, gateway proxy để chặn đường gọi phù hợp và service policy để đảm bảo backend là nguồn quyết định cuối cùng.

4.6 Thiết kế preview/download authorization

Download được thiết kế thành luồng hai bước. Ở bước đầu, frontend gọi metadata-service thông qua gateway để xin authorize. Metadata-service kiểm tra policy và nếu hợp lệ sẽ cấp grant token ngắn hạn. Ở bước sau, frontend dùng grant token để gọi document-service. Document-service xác minh grant token hoặc tái authorize trước khi cấp presigned URL hoặc stream endpoint.



Hình 4.3: Sequence cấp quyền download và stream file

Với tài liệu **PUBLIC** hoặc **INTERNAL**, nếu policy cho phép, hệ thống có thể trả về presigned URL. Với tài liệu **CONFIDENTIAL** hoặc **SECRET**, hệ thống trả về `url: null`, `watermarkRequired: true` và `streamingEndpoint`. Cách này giúp document-service kiểm soát đường tải nội dung nhạy cảm tốt hơn.

4.7 Malware scanning và DLP

Document-service thực hiện scanning trước khi ghi object vào MinIO. Với payload EICAR trong môi trường demo, upload bị chặn và không tạo object cũng như không đăng ký metadata version. Đây là một điểm quan trọng: nếu scan chỉ phát hiện sau khi file đã lưu, hệ thống sẽ phải xử lý cleanup phức tạp hơn.

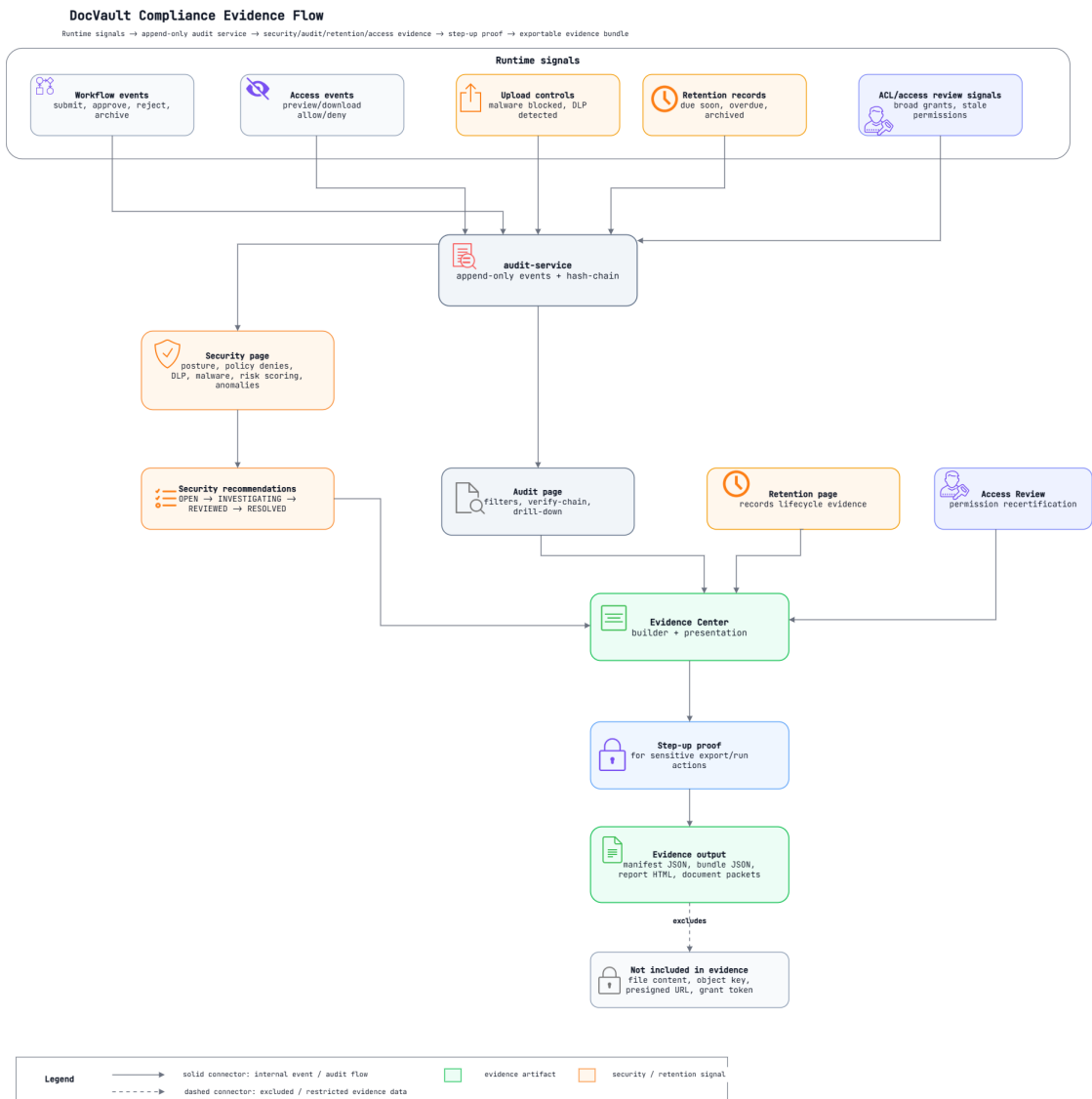
DLP scan phát hiện một số pattern nhạy cảm như email, số điện thoại/số định danh và từ khóa như **secret**, **confidential**, **internal only**. Khi phát hiện dữ liệu nhạy cảm trong tài liệu public/internal, hệ thống nâng classification lên **CONFIDENTIAL**. Người dùng không có quyền không thể hạ classification xuống public/internal nếu không có lý do override hợp lệ.

4.8 Audit ingest boundary

Audit-service không cho phép người dùng thường append audit event trực tiếp. Các service tin cậy phải gửi header service token **x-docvault-service-token**. Token này được cấu hình bằng **AUDIT_INGEST_TOKEN**. Điều này ngăn người dùng có JWT hợp lệ tạo audit event giả để che giấu hoặc làm sai lệch lịch sử.

4.9 Luồng compliance evidence

Compliance evidence trong DocVault được xây dựng từ nhiều tín hiệu runtime: workflow event, access event, upload control, retention record và access review signal. Các tín hiệu này được audit-service ghi nhận theo dạng append-only và hash-chain, sau đó được các màn hình Security, Audit, Retention, Access Review và Evidence Center tổng hợp thành bằng chứng có thể xuất ra ngoài.



Hình 4.4: Luồng tổng hợp compliance evidence trong DocVault

Điểm cần nhấn mạnh là evidence output không chứa file content, object key, presigned URL hoặc grant token. Nhờ đó Compliance Officer có thể kiểm tra tính tuân thủ và truy vết hoạt động mà không được mở rộng thành quyền truy cập nội dung tài liệu.

4.10 API contract tổng quan

API của hệ thống được expose qua gateway với base path `/api`. Các nhóm endpoint chính gồm:

Bảng 4.2: Nhóm API chính của DocVault

Nhóm API	Ví dụ endpoint	Mục đích
Auth	/auth/login, /auth/me, /auth/logout	Đăng nhập, callback, logout và lấy người dùng hiện tại.
Metadata	/metadata/documents, /metadata/documents/:id	Tạo, đọc, cập nhật metadata tài liệu.
ACL	/metadata/documents/:id/acl	Thêm/xem rule ACL theo tài liệu.
Version	/metadata/documents/:id/versions	Đăng ký version metadata sau upload.
Download authorization	/metadata/documents/:id/download-authorize	Xin quyền download và grant token.
Preview authorization	/metadata/documents/:id/preview-authorize	Xin quyền preview nội dung file.
Document blob	/documents/:id/upload /documents/:id/presign-download /documents/:id/versions/:v/stream	Upload, presign hoặc stream file.
Workflow	/workflow/:id/submit, /workflow/:id/approve	Chuyển trạng thái tài liệu.
Audit	/audit/query, /audit/verify-chain, /audit/security-summary	Truy vấn audit và kiểm tra hash-chain.
Evidence	/metadata/documents/:id/evidence-packet	Xuất evidence packet metadata-only.
Retention	/metadata/retention/documents /metadata/retention/run	Xem và chạy retention demo.

Chương 5

Triển khai ứng dụng DocVault

5.1 Cấu trúc monorepo

DocVault được tổ chức dưới dạng monorepo sử dụng pnpm và Turbo. Cách tổ chức này giúp quản lý frontend, backend services, shared libraries, infrastructure và tài liệu trong cùng một repository. Các thư mục chính gồm:

- `apps/web`: ứng dụng frontend Next.js.
- `services/gateway`: API Gateway.
- `services/metadata-service`: service quản lý metadata, ACL, policy và retention.
- `services/document-service`: service xử lý upload/download file và MinIO/S3.
- `services/workflow-service`: service xử lý state machine tài liệu.
- `services/audit-service`: service ghi/truy vấn audit log và verify hash-chain.
- `services/notification-service`: notification sink cho môi trường demo/dev.
- `libs/auth`, `libs/contracts`, `libs/throttler`: các thư viện dùng chung.
- `infra`: Docker Compose, Kubernetes, Helm, Argo CD, Terraform và các cấu hình hạ tầng.
- `docs`: tài liệu thiết kế, hướng dẫn chạy, evidence và screenshot.

Monorepo giúp báo cáo, code, pipeline và evidence nằm chung một vòng đời. Khi một thay đổi ảnh hưởng đến API contract hoặc security policy, nhóm phát triển có thể cập nhật code, test và tài liệu trong cùng một pull request.

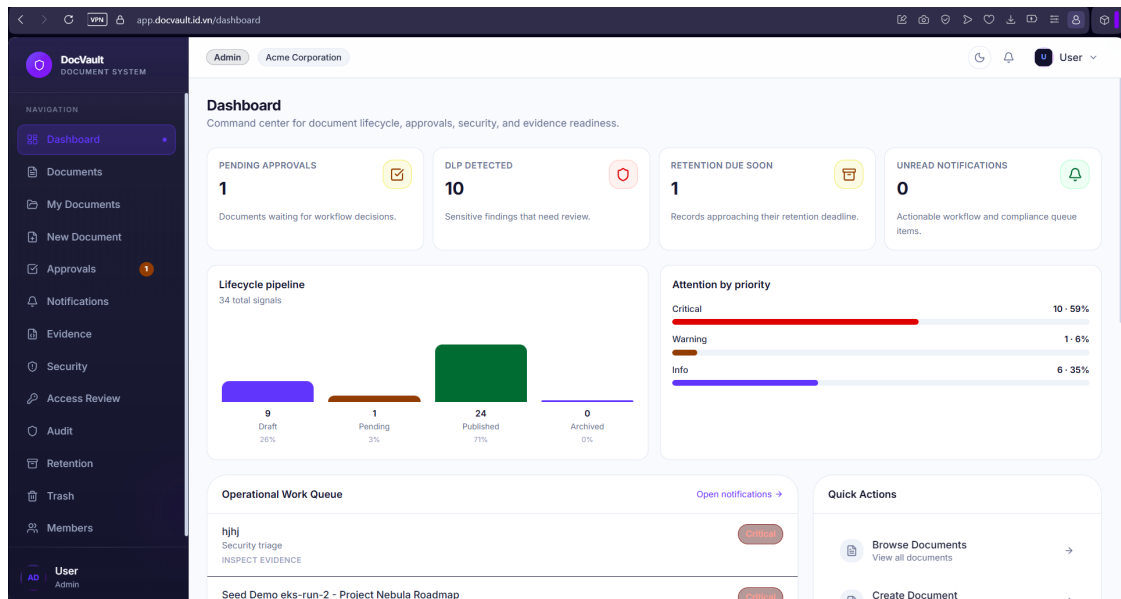
5.2 Frontend Next.js

Frontend được xây dựng bằng Next.js, React, TanStack Query, React Hook Form, Zod, Tailwind CSS và Radix UI. Ứng dụng sử dụng App Router và tổ chức theo các nhóm route đã đăng nhập hoặc chưa đăng nhập. Frontend gọi gateway qua biến môi trường

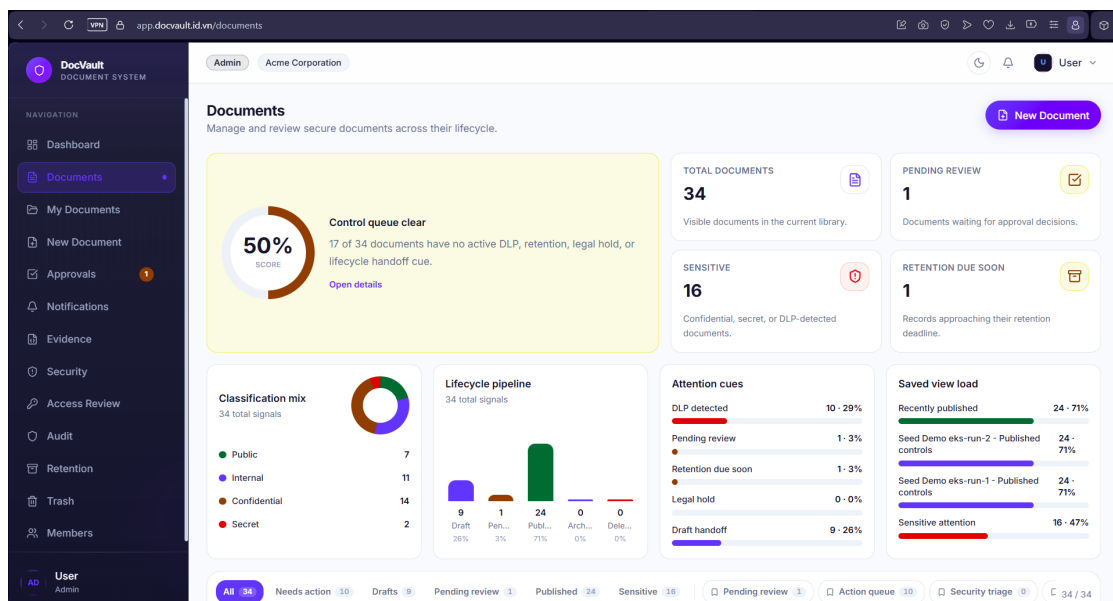
NEXT_PUBLIC_API_BASE_URL, mặc định trỏ đến `http://localhost:3000/api` trong môi trường local.

Frontend không chỉ là lớp hiển thị danh sách tài liệu. Ứng dụng đã bao phủ nhiều luồng vận hành quan trọng:

- Đăng nhập SSO qua Keycloak và quản lý phiên.
- Dashboard tổng quan tài liệu, pending approval, DLP, retention và readiness.
- Documents và My Documents với lọc nâng cao, quick views, saved views, folder tree và phân trang.
- New Document và Edit Document cho tạo/chỉnh metadata và upload file.
- Document Detail với preview/download, version history, workflow timeline, ACL, comments, activity, share link, legal hold, DLP findings, AI guardrails và evidence links.
- Approvals với SLA summary, review drawer, approve/reject và reason preset.
- Audit, Security, Evidence Center, Retention và Access Review cho compliance/admin.
- Notifications, Trash, Members, Profile, Settings và Demo Kit.



Hình 5.1: Màn hình dashboard của DocVault



Hình 5.2: Màn hình danh sách tài liệu

5.2.1 Permission-aware UI

Frontend có helper phân quyền để xác định action nào nên hiển thị, action nào nên bị disable và lý do deny là gì. Ví dụ, Compliance Officer có thể thấy lý do bị chặn download thay vì chỉ không thấy nút. Tuy nhiên, helper frontend không được xem là boundary bảo mật cuối cùng. Backend vẫn là nơi quyết định policy. Điều này tránh tình trạng người dùng tự gọi API để vượt qua UI.

5.3 Gateway

Gateway là entry point của hệ thống. Service này xác thực JWT từ Keycloak, kiểm tra role ở mức route, expose health endpoint, proxy request xuống service phía sau và truyền các header ngữ cảnh. Các header quan trọng gồm:

- authorization
- x-request-id
- x-user-id
- x-roles
- x-groups

Gateway proxy các nhóm route chính như /metadata, /documents, /workflow, /audit và /notify. Nhờ đó frontend chỉ cần biết một API base URL, trong khi bên trong hệ thống vẫn tách service theo bounded context.

5.4 Metadata-service

Metadata-service là nguồn sự thật cho tài liệu logic và policy. Service này sở hữu các dữ liệu và nghiệp vụ:

- document record;
- classification và tags;
- current version pointer;
- ACL entries;
- workflow history;
- download/preview authorization;
- retention evidence;
- access impact simulation;
- AI guardrails metadata-only.

Metadata-service là nơi quan trọng nhất để enforce policy liên quan đến metadata và grant token. Khi document-service cần đăng ký version mới hoặc khi workflow-service cần cập nhật trạng thái, các service này gọi metadata-service thay vì ghi trực tiếp vào database.

5.5 Document-service

Document-service xử lý phần blob file. Service này nhận upload multipart, tính checksum SHA-256, scan malware/DLP, sinh object key và upload file lên MinIO/S3. Sau khi upload thành công, service gọi metadata-service để đăng ký version mới.

Đối với download, document-service không cấp file một cách độc lập. Service xác minh grant token hoặc tái authorize thông qua metadata-service trước khi trả presigned URL hoặc stream endpoint. Với tài liệu nhạy cảm, service trả về stream endpoint để áp dụng posture kiểm soát tốt hơn.

5.6 Workflow-service

Workflow-service sở hữu logic chuyển trạng thái. Service đọc trạng thái hiện tại từ metadata-service, kiểm tra transition hợp lệ rồi yêu cầu metadata-service cập nhật trạng thái. Các transition chính:

- SUBMIT: DRAFT -> PENDING
- APPROVE: PENDING -> PUBLISHED
- REJECT: PENDING -> DRAFT
- ARCHIVE: PUBLISHED -> ARCHIVED

Sau mỗi transition, workflow-service phát audit event và gọi notification-service. Cách này giúp workflow vẫn là nơi điều phối nghiệp vụ, nhưng metadata-service vẫn là owner dữ liệu trạng thái.

5.7 Audit-service

Audit-service ghi và truy vấn audit log. Service hỗ trợ append audit event từ các service tin cậy, query theo actor/action/resource/result/time, verify hash-chain và security summary. Endpoint audit query chỉ dành cho Compliance Officer hoặc Admin.

Hash-chain giúp phát hiện sửa đổi audit event ngoài luồng. Nếu một event cũ bị chỉnh trực tiếp trong database, verify-chain trả về trạng thái không hợp lệ. Trong phạm vi đề án, đây là cơ chế tamper-evident phù hợp để chứng minh ý tưởng kiểm soát audit.

5.8 Notification-service

Notification-service hiện là notification sink cho môi trường dev/demo. Service nhận payload từ workflow hoặc các sự kiện liên quan và lưu/log để frontend hiển thị notification center. Đây chưa phải hệ thống email, SMS hoặc push notification hoàn chỉnh, nhưng đủ để mô phỏng luồng thông báo trong sản phẩm.

5.9 Hạ tầng local

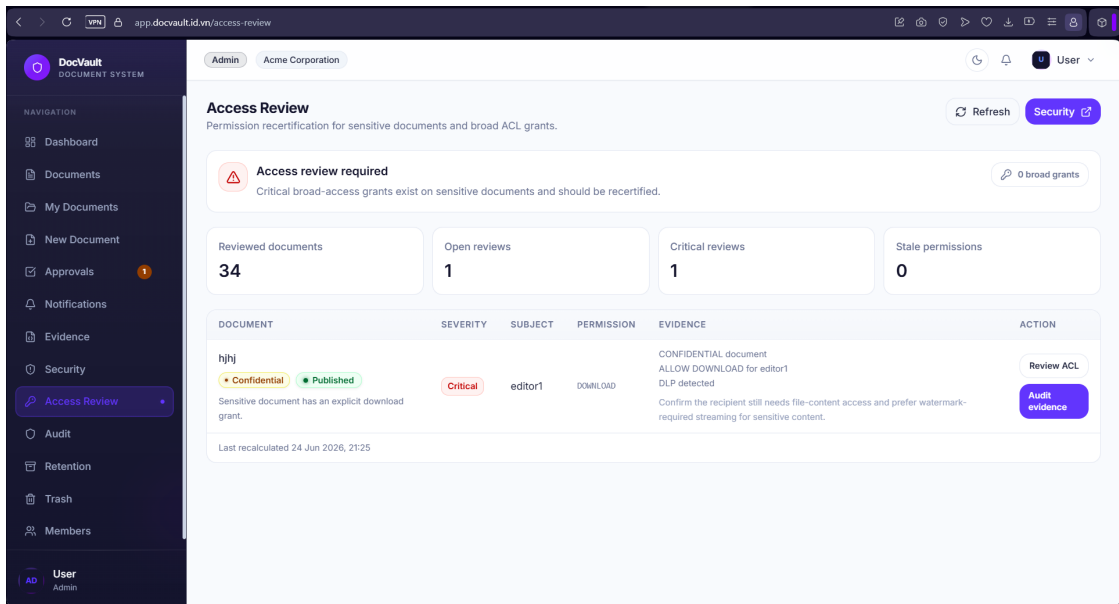
Môi trường local sử dụng Docker Compose để chạy các dependency:

- PostgreSQL cho metadata.
- MongoDB cho audit.
- MongoDB Express để quan sát dữ liệu audit trong dev.
- MinIO cho object storage.
- MinIO init job để tạo bucket và bật cấu hình cần thiết.
- Keycloak cho identity, realm, roles và demo users.

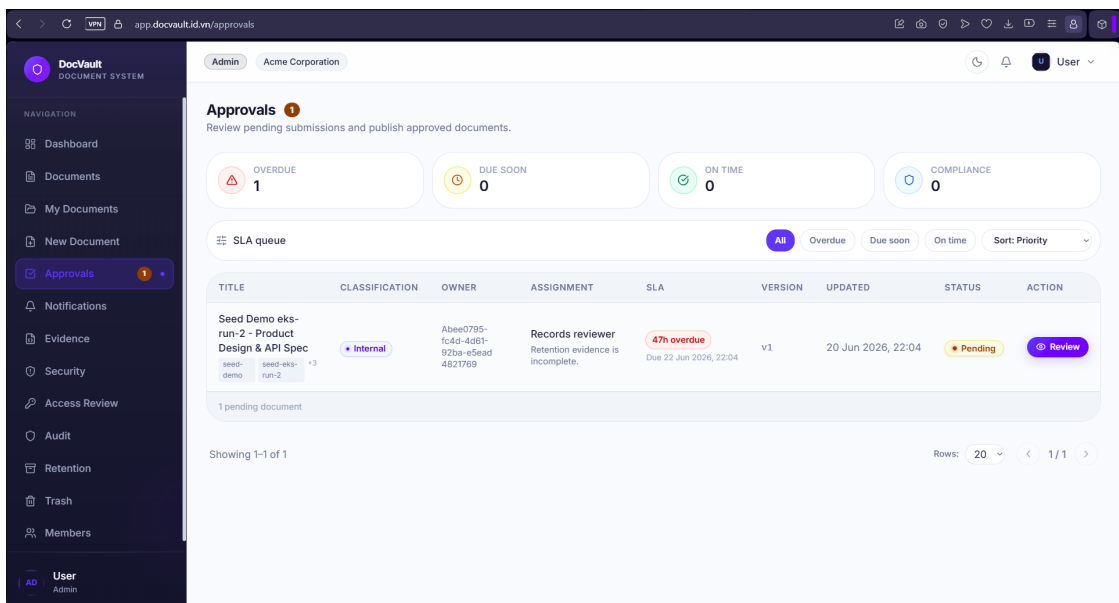
Các service backend chạy trên các port 3000 đến 3005, frontend thường chạy trên 3100 để tránh xung đột với gateway.

5.10 Các màn hình minh họa

Các hình dưới đây thể hiện một số giao diện chính đã triển khai trong frontend.



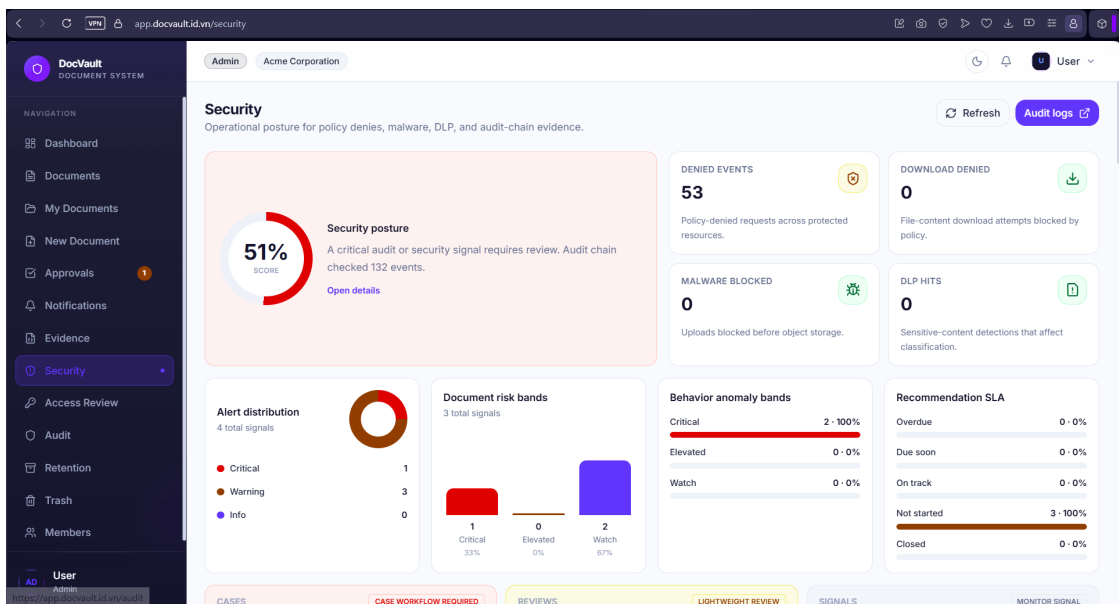
Hình 5.3: Màn hình rà soát quyền truy cập



Hình 5.4: Màn hình phê duyệt tài liệu

TIMESTAMP	ACTOR	ROLES	ACTION	RESOURCE TYPE	RESOURCE ID	RESULT	REASON	TRACE ID
24 Jun 2026, 21:22	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 21:14	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 15:30	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 00:30	Admin One	admin	DOCUMENT_ACL_UPDATED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	5146dc..1422fa
23 Jun 2026, 00:29	Admin One	admin	COMMENT_ADDED	COMMENT	a3eb59c..64af359	SUCCESS	-	b8465b..994f65
23 Jun 2026, 00:29	Admin One	admin	DOCUMENT_DOWNLOAD_AUTHORIZED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	06348b..6626b3
23 Jun 2026, 00:28	Admin One	admin	DOCUMENT_PREVIEW_AUTHORIZED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	171f66..6626b3
23 Jun 2026, 00:28	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
21 Jun 2026, 01:18	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	637a15..25b29d
21 Jun 2026, 01:18	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	11fc07..b709c8
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	fd43b2..78aa20
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	6e5614..9afcab
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	19146d..79eb82

Hình 5.5: Màn hình audit log



Hình 5.6: Màn hình security posture

5.11 Luồng demo ứng dụng

Luồng demo chính được thiết kế để chứng minh cả nghiệp vụ và bảo mật:

1. Editor đăng nhập, tạo tài liệu và upload file.
2. Editor submit tài liệu để chuyển từ Draft sang Pending.
3. Approver đăng nhập, mở hàng đợi Approvals và approve tài liệu.
4. Viewer đăng nhập, xem tài liệu đã Published và download nếu policy cho phép.

5. Compliance Officer đăng nhập, mở Audit/Security/Evidence nhưng bị deny pre-view/download file content.

Luồng này thể hiện đủ các thành phần chính của hệ thống: identity, gateway, metadata, document storage, workflow, audit, notification và frontend.

Chương 6

Quy trình DevSecOps và hạ tầng triển khai

6.1 Mục tiêu của DevSecOps pipeline

Pipeline DevSecOps của DocVault được thiết kế để tự động hóa các bước từ mã nguồn đến triển khai, đồng thời tích hợp các kiểm soát bảo mật vào từng giai đoạn. Mục tiêu không chỉ là build được ứng dụng, mà còn là tạo bằng chứng cho thấy mã nguồn, phụ thuộc, image, cấu hình hạ tầng và ứng dụng sau triển khai đã được kiểm tra ở mức phù hợp.

Các mục tiêu cụ thể của pipeline gồm:

- Cài đặt phụ thuộc và kiểm tra môi trường build.
- Chạy unit test và kiểm tra chất lượng mã nguồn.
- Chạy SAST, SCA, filesystem scan và IaC scan trước khi build image.
- Build Docker image cho frontend và các service backend.
- Quét Docker image trước khi push registry.
- Push image và cập nhật Helm values trên GitOps branch.
- Để Argo CD đồng bộ trạng thái mong muốn lên Kubernetes/EKS.
- Kiểm tra health của Argo CD application và smoke test ứng dụng đã deploy.
- Chạy DAST bằng OWASP ZAP trên web base URL.
- Archive report và cleanup workspace.

6.2 Jenkins pipeline tổng thể

Pipeline chính nằm trong `Jenkinsfile`. Shared library được lấy từ thư viện DocVault trên branch DevSecOps pipeline; cấu hình chung nằm trong `vars/docvaultConfig.groovy`. Pipeline chạy trên Jenkins agent có Docker, Git, kubectl và các công cụ cần thiết.

Các tham số quan trọng gồm:

- **FORCE_BUILD_ALL**: build lại toàn bộ service dù change detection không yêu cầu.
- **GITOPS_BRANCH**: branch chứa Helm values để Argo CD sync.
- **DEPLOY_TARGET_URL**: URL web sau deploy để smoke test.
- **RUN_ARGO_HEALTH_CHECK**: bật/tắt kiểm tra Argo CD health.
- **RUN_ZAP**: bật/tắt DAST bằng OWASP ZAP.
- **ZAP_TARGET**: URL target cho ZAP baseline scan.

Listing 6.1: Luồng DevSecOps pipeline của DocVault

```
Checkout -> Prevent Loop -> System Check -> Install
-> Pre-build Security (parallel)
    SCA | Trivy FS | Unit Tests | SonarQube | Checkov | Terraform Validate
-> Build & Scan Services
-> Push & GitOps
-> Argo CD Health Check
-> Post-deploy Smoke Test
-> DAST - OWASP ZAP
-> Post Cleanup
```

6.3 Pre-build Security

Stage Pre-build Security chạy song song các bước độc lập. Việc chạy song song giúp giảm thời gian pipeline nhưng vẫn giữ đầy đủ kiểm soát. Các bước gồm:

- **SCA - Dependency Check**: phân tích dependency để phát hiện lỗ hổng thư viện.
- **Trivy FS Scan**: quét filesystem repository.
- **Unit Tests**: chạy test của các package/service.
- **SAST - SonarQube**: phân tích chất lượng và bảo mật mã nguồn [5].
- **IaC - Checkov Scan**: quét Terraform/Kubernetes/IaC để phát hiện cấu hình rủi ro [6].
- **IaC - Terraform Validate**: kiểm tra cú pháp và tính hợp lệ của Terraform.

Việc đặt các bước này trước build image giúp phát hiện lỗi sớm. Ví dụ, nếu dependency có lỗ hổng nghiêm trọng hoặc Terraform không validate được, pipeline có thể dừng trước khi tốn thời gian build/push image.

6.4 Build và scan image

Stage **Build & Scan Services** build Docker image cho các service chính và frontend. Cấu hình trong `docvaultConfig.groovy` liệt kê các service backend: gateway, metadata-service, document-service, notification-service, workflow-service và audit-service. Frontend được build từ `apps/web/Dockerfile`; backend dùng `Dockerfile.backend`.

Sau khi build, image được scan bằng Trivy. Trivy hỗ trợ quét container image, filesystem, lockfile và các loại artifact khác để phát hiện lỗ hổng, misconfiguration hoặc secret tùy cấu hình [7]. Với DocVault, image scan đóng vai trò gate trước khi push image lên registry.

6.5 Push image và GitOps update

Khi có service được build hoặc infra thay đổi, stage **Push & GitOps** thực hiện push image và cập nhật Helm values trong GitOps branch. Thay vì triển khai trực tiếp từ Jenkins vào cluster, pipeline cập nhật source of truth trong Git. Argo CD sau đó đọc branch GitOps và đồng bộ cluster.

Mô hình này có ba lợi ích. Thứ nhất, lịch sử triển khai được lưu trong Git commit. Thứ hai, Jenkins không cần sở hữu toàn bộ logic reconcile trạng thái Kubernetes. Thứ ba, có thể rollback bằng cách quay lại commit hoặc tag tương ứng trong GitOps branch.

6.6 Argo CD health check

Nếu `RUN_ARGO_HEALTH_CHECK=true`, pipeline chạy stage kiểm tra Argo CD application. Các application chính gồm gateway, metadata, document-service, workflow-service, audit-service, notification-service và web. Pipeline chờ đến khi các application đạt trạng thái **Synced** và **Healthy** trong thời gian timeout cấu hình.

Argo CD đóng vai trò GitOps controller cho Kubernetes. Nó so sánh trạng thái mong muốn trong Git với trạng thái thực tế trong cluster, sau đó sync khi cần [4]. Với DocVault, Argo CD giúp quá trình triển khai lên EKS có tính lặp lại và quan sát được.

6.7 Post-deploy smoke test

Sau khi deploy, pipeline có thể chạy smoke test trên `DEPLOY_TARGET_URL`. Stage này kiểm tra:

- GET `<DEPLOY_TARGET_URL>`
- GET `<DEPLOY_TARGET_URL>/api/health`

Kết quả kỳ vọng là HTTP 2xx hoặc 3xx cho web root và 2xx cho health endpoint. Smoke test không thay thế test nghiệp vụ đầy đủ, nhưng là gate nhanh để đảm bảo deployment cơ bản đã hoạt động.

6.8 DAST bằng OWASP ZAP

Khi `RUN_ZAP=true`, pipeline chạy OWASP ZAP baseline scan trên web base URL. ZAP baseline scan phù hợp cho pipeline vì có thể crawl nhẹ ứng dụng và sinh report HTML/JSON [8]. Trong cấu hình DocVault, cảnh báo low/medium được archive làm bằng chứng, trong khi high/critical findings cần được xem là action item hoặc fail pipeline tùy cấu hình.

Artifacts quan trọng gồm:

- `zap-report/zap_report.html`
- `zap-report/zap_report.json`

6.9 Observability

DocVault có cấu hình observability qua các Argo CD application cho Prometheus, Grafana, Loki và promtail. Prometheus/Grafana phục vụ metric và dashboard; Loki/promtail phục vụ log tập trung từ Kubernetes namespace `docvault`. Các bằng chứng nên chụp gồm pod health, CPU/RAM, workload status và Grafana Explore với query `{namespace="docvault"}`.

6.10 Mapping stage, công cụ và bằng chứng

Bảng 6.1: Mapping pipeline stage với công cụ và bằng chứng

Stage	Công cụ	Bằng chứng/artifact	Điều kiện fail chính
Install	pnpm	Log cài đặt phụ thuộc	Dependency install lỗi
Unit Tests	Jest/Vitest	Test report/log	Test fail
SAST	SonarQube	Quality port/dashboard	Quality gate hoặc lỗi scanner
SCA	Dependency Check	Dependency report	Vulnerability vượt ngưỡng

Stage	Công cụ	Bảng chứng/artifact	Điều kiện fail chính
Trivy FS	Trivy	FS scan report/log	Lỗi hỏng hoặc cấu hình lỗi vượt ngưỡng
IaC Scan	Checkov	Checkov report/log	Rule nghiêm trọng fail
Terraform Vali-date	Terraform	Validate log	Terraform không hợp lệ
Build image	Docker	Image tag/build log	Build lỗi
Image scan	Trivy	Image scan report/log	CVE vượt ngưỡng
GitOps update	Git/Helm values	Commit trên GitOps branch	Push/update lỗi
Argo CD health	kubectl/Argo CD	Synced/Healthy status	App không healthy trong timeout
Smoke test	HTTP client	HTTP status log	Web/health không reachable
DAST	OWASP ZAP	HTML/JSON report	High/Critical finding hoặc target unreachable

6.11 Trạng thái hiện tại của pipeline

Theo tài liệu [docs/pipeline-hardening-summary.md](#), Jenkins pipeline đã có build chạy xanh toàn bộ ở build #61 trên branch `devsecops-pipeline`. Các stage đã pass gồm Pre-build Security, Build & Scan Services, Push & GitOps, Argo CD Health Check, Post-deploy Smoke Test và DAST OWASP ZAP.

Tuy nhiên, báo cáo vẫn trình bày pipeline ở trạng thái đang hoàn thiện vì một pipeline DevSecOps production-ready cần thêm nhiều năng lực: SBOM, ký image/artifact, provenance, policy-as-code admission, quản lý secret/KMS chặt hơn, alerting hoàn chỉnh, roll-back runbook và evidence retention có cấu trúc. Những nội dung này được xem là hướng phát triển tiếp theo.

Chương 7

Kiểm thử, đánh giá và bằng chứng

7.1 Chiến lược kiểm thử

Kiểm thử DocVault được tổ chức theo nhiều lớp. Ở lớp code, các service backend có unit test cho policy, controller, security guard và các helper quan trọng. Frontend có test cho permission helper, form, component và một số behavior UI. Ở lớp runtime, script E2E kiểm tra các luồng nghiệp vụ và bảo mật qua API thật. Ở lớp pipeline, Jenkins chạy các bước test/scan/build/deploy/smoke/DAST để kiểm tra toàn bộ vòng đời phát hành.

Chiến lược này phù hợp với đặc thù của hệ thống bảo mật. Một unit test có thể kiểm tra một rule policy cụ thể, nhưng không chứng minh được toàn bộ luồng từ frontend đến gateway, metadata-service, document-service và audit-service. Ngược lại, E2E runtime có thể chứng minh luồng chính hoạt động, nhưng khó bao phủ mọi nhánh policy nhỏ. Vì vậy, hệ thống cần kết hợp nhiều loại kiểm thử.

7.2 Kiểm thử runtime

Theo docs/verification-report.md, lệnh E2E runtime chính là:

Listing 7.1: Lệnh kiểm thử E2E runtime

```
pnpm test:e2e
```

Kết quả gần nhất được ghi nhận trong tài liệu là **All required E2E checks passed**. Các nhóm kiểm thử runtime gồm:

- Auth boundary: request thiếu token và token hết hạn bị từ chối.
- Create policy: viewer không được tạo tài liệu, editor được tạo.
- Metadata policy: truy cập trái phép vào confidential document bị từ chối ở detail, history, comments và ACL.
- GROUP ACL: group `finance-team` được normalize và kiểm tra đúng.
- Confidential download posture: viewer bị deny, editor nhận stream-only response.

- Malware scan: EICAR upload bị chặn.
- DLP: upload có dữ liệu nhạy cảm bị phát hiện và nâng classification.
- Workflow: submit, approve và duplicate approve conflict.
- Retention: approval stamp retention, retention job archive due record.
- Compliance Officer: được phép xem metadata, audit, retention; bị chặn preview, presign, stream và download.
- Audit: query, verify-chain, security summary và audit ingest boundary.

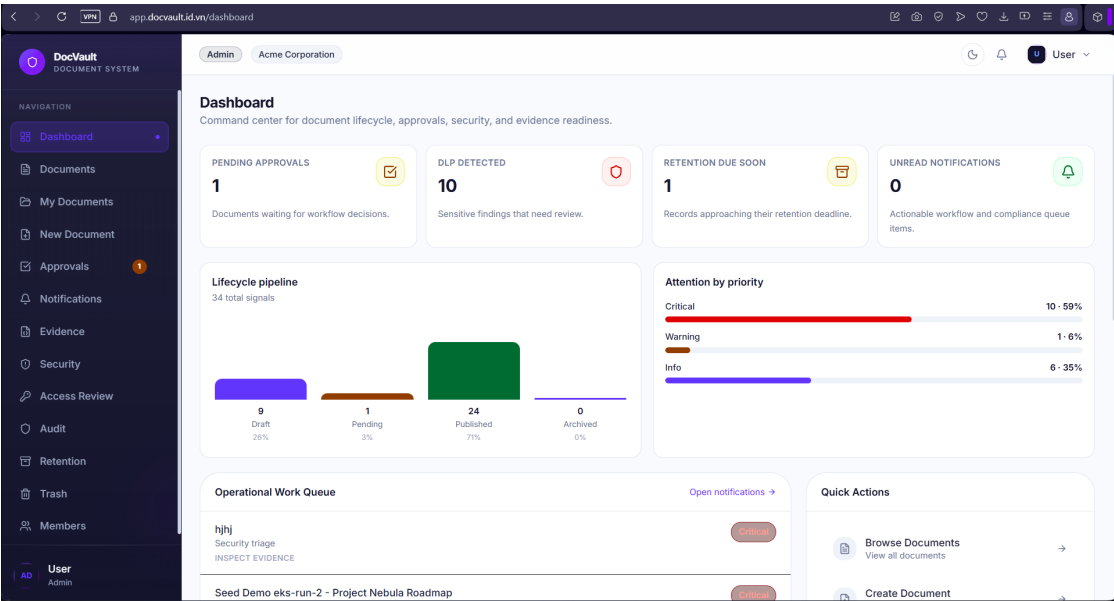
Bảng 7.1: Tóm tắt bằng chứng kiểm thử runtime

Khu vực	Bằng chứng	Kết quả
Authentication	Missing/expired token rejected	PASS
Authorization	Viewer create denied, editor create allowed	PASS
Metadata policy	Viewer guessed confidential access denied	PASS
ACL	GROUP READ ACL metadata access	PASS
Download posture	Confidential presign withheld direct URL	PASS
Malware	EICAR upload blocked before object/version creation	PASS
DLP	Sensitive upload escalated classification	PASS
Workflow	Submit/approve/duplicate approve conflict	PASS
Retention	Auto-archive due record and workflow history	PASS
Compliance boundary	Compliance file content access denied	PASS
Audit	Verify-chain and security summary available	PASS
Audit ingest	Viewer cannot ingest fake audit event	PASS

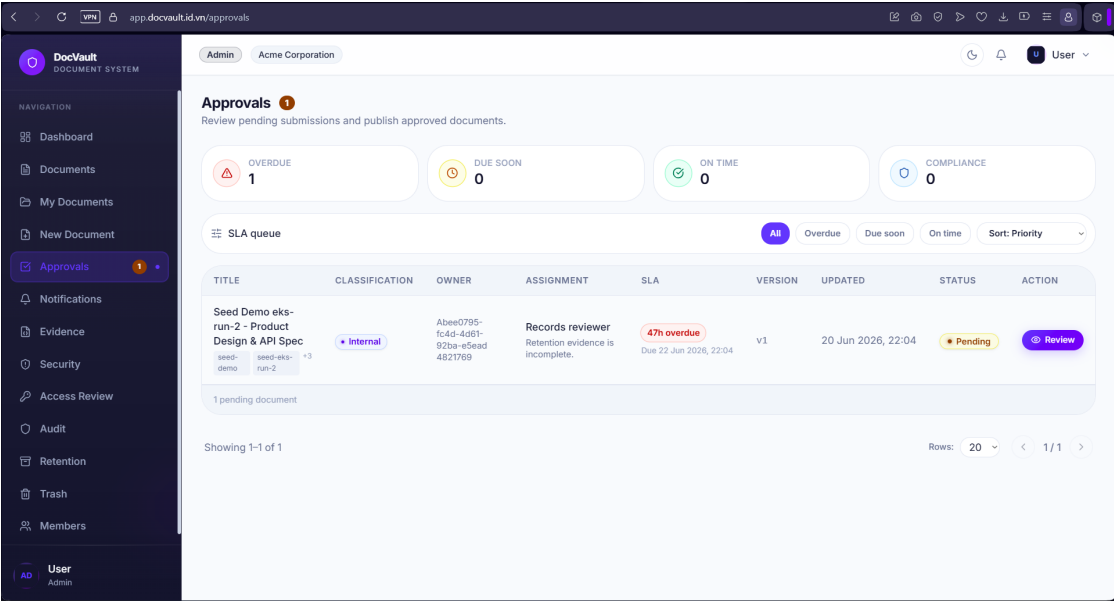
7.3 Kiểm thử frontend và UI evidence

Frontend được kiểm tra bằng lint, build, typecheck và test component/helper. Ngoài ra, bộ screenshot trong `report/images/screen` ghi nhận các màn hình trọng yếu như Dashboard, Documents, Approvals, Audit, Security và Access Review trên môi trường triển khai. Các screenshot này vừa minh họa UI, vừa cung cấp bằng chứng trực quan

rằng các màn hình chính render được và hiển thị trạng thái dữ liệu phù hợp.



Hình 7.1: Bảng chứng giao diện dashboard command center



Hình 7.2: Bảng chứng giao diện hàng đợi phê duyệt và trạng thái SLA

7.4 Security evidence

Tài liệu docs/web-security-evidence.md tổng hợp các kiểm soát bảo mật runtime. Các điểm nổi bật gồm:

- Compliance Officer bị chặn preview, stream, presign và download file content.
- Metadata access boundary được áp dụng cho detail, workflow history, comments và ACL list.

- Frontend permission helper được align với backend policy ở mức UI hint.
- Grant token cho download/preview có TTL 300 giây và hỗ trợ rotation bằng `kid`.
- MinIO bucket local bật SSE-S3 trong init script.
- Malware scanning chặn EICAR trước khi ghi object.
- DLP phát hiện pattern nhạy cảm và nâng classification.
- Audit ingest cần service token.
- Audit verify-chain và security summary được expose qua gateway.
- Evidence packet không chứa file content, presigned URL, grant token hoặc object key.

7.5 Pipeline evidence

Pipeline evidence gồm log và artifact từ Jenkins. Theo tài liệu hardening summary của pipeline, build #61 trên branch `devsecops-pipeline` đã pass các stage chính: Pre-build Security, Build & Scan Services, Push & GitOps, Argo CD Health Check, Post-deploy Smoke Test và DAST OWASP ZAP.

Các bằng chứng nên được đưa vào phụ lục hoặc slide bảo vệ:

- Screenshot Jenkins build xanh và stage view.
- SonarQube report hoặc quality gate.
- Dependency Check report.
- Trivy filesystem/image scan log.
- Checkov/Terraform validate log.
- Docker image tags được push.
- GitOps commit cập nhật Helm values.
- Argo CD application `Synced/Healthy`.
- Smoke test log cho web root và `/api/health`.
- ZAP HTML/JSON report.
- Grafana/Loki screenshot cho observability.

7.6 Đánh giá theo yêu cầu

Bảng 7.2: Mapping yêu cầu với hiện thực và bằng chứng

Yêu cầu	Hiện thực	Bằng chứng
Đăng nhập và phân quyền	Keycloak, JWT, gateway role guard, frontend permission helper	README, API contract, E2E auth checks
Quản lý vòng đời tài liệu	Documents, versions, workflow history, workflow-service transitions	Project status, E2E workflow checks
Kiểm soát download	Metadata authorization, grant token, document-service pre-sign/stream	web-security-evidence, E2E confidential posture
Compliance boundary	Compliance meta-data/audit/evidence allowed, file content denied	web-security-evidence, E2E compliance checks
Audit tamper-evident	audit-service hash-chain, verify-chain endpoint	verification report, audit docs
Malware/DLP	Malware scanner, DLP scanner, classification escalation	E2E malware/DLP checks
Retention evidence	Retention fields, retention run, audit event	E2E retention checks
DevSecOps pipeline	Jenkins stages, security scans, GitOps, Argo CD, smoke, ZAP	pipeline-hardening-summary, Jenkins-file

7.7 Hạn chế hiện tại

Mặc dù hệ thống đã có nhiều bằng chứng runtime, vẫn còn một số hạn chế:

- Notification-service hiện vẫn là dev sink, chưa phải hệ thống email/webhook/push hoàn chỉnh.
- Browser-level E2E chưa phải bằng chứng chính cho toàn bộ luồng nghiệp vụ; phần mạnh nhất hiện là API/runtime E2E.
- Shared auth/contracts còn có thể tiếp tục consolidate để giảm trùng lặp giữa service.
- Pipeline đã có nhiều stage, nhưng cần bổ sung SBOM, artifact signing, provenance và policy-as-code để tiến gần production-grade DevSecOps.
- Secret/KMS/network policy và backup/restore cần được hardening thêm nếu triển khai production thật.

7.8 Nhận xét chung

DocVault đã đạt được mục tiêu của một hệ thống quản lý tài liệu bảo mật ở mức độ chuyên ngành: có kiến trúc rõ, có luồng nghiệp vụ hoàn chỉnh, có kiểm soát bảo mật backend, có audit/evidence và có pipeline DevSecOps với nhiều bước kiểm tra tự động. Điểm quan trọng là báo cáo không chỉ mô tả ý tưởng mà có thể đối chiếu với mã nguồn, tài liệu evidence và pipeline hiện có trong repository.

Chương 8

Kết luận và hướng phát triển

8.1 Kết quả đạt được

Đề tài đã xây dựng được hệ thống DocVault theo hướng quản lý tài liệu bảo mật và triển khai DevSecOps. Về mặt ứng dụng, hệ thống đã có frontend Next.js với nhiều màn hình nghiệp vụ, API Gateway, các microservice NestJS và hạ tầng local gồm Keycloak, PostgreSQL, MongoDB và MinIO. Các luồng chính như tạo tài liệu, upload file, gửi duyệt, phê duyệt, xem danh sách, xem chi tiết, preview/download, audit, security dashboard, evidence center và retention đã được triển khai.

Về mặt bảo mật runtime, hệ thống đã thể hiện rõ nhiều kiểm soát quan trọng. RBAC được dùng cho quyền theo vai trò; ACL được dùng cho quyền theo tài liệu; policy backend quyết định truy cập metadata và nội dung file; Compliance Officer bị giới hạn ở metadata/audit/evidence; tài liệu confidential/secret có posture stream-only; audit-service có hash-chain; malware scanner chặn EICAR; DLP phát hiện dữ liệu nhạy cảm và nâng classification; evidence packet không làm lộ nội dung file hoặc token nhạy cảm.

Về mặt DevSecOps, đề tài đã thiết kế và triển khai Jenkins pipeline có các stage kiểm thử và quét bảo mật. Pipeline tích hợp SCA, SAST, Trivy filesystem/image scan, Checkov IaC scan, Terraform validate, unit tests, Docker build, GitOps update, Argo CD health check, smoke test và OWASP ZAP DAST. Đây là nền tảng tốt để tiếp tục hoàn thiện quy trình phát hành an toàn cho DocVault.

8.2 Mức độ đáp ứng mục tiêu

Các mục tiêu chính của đề án đã được đáp ứng ở mức phù hợp với môn Đồ Án Chuyên Ngành:

- Hệ thống có kiến trúc microservices và service boundary rõ ràng.
- Ứng dụng web đã bao phủ phần lớn user flow quan trọng.
- Chính sách bảo mật được enforce ở backend thay vì chỉ ẩn/hiện ở frontend.

- Có bằng chứng runtime cho các luồng quan trọng: auth, RBAC/ACL, confidential download posture, malware, DLP, retention, audit và compliance deny.
- Pipeline DevSecOps đã có cấu trúc và nhiều security gate quan trọng.
- Tài liệu kỹ thuật, API contract, evidence và hướng dẫn demo được lưu trong repository.

8.3 Hạn chế

Do phạm vi và thời gian của đồ án, hệ thống vẫn còn một số hạn chế. Thứ nhất, notification-service hiện mới ở mức dev sink, chưa tích hợp email, webhook hoặc push notification thật. Thứ hai, pipeline DevSecOps tuy đã có nhiều stage nhưng vẫn cần hardening thêm để đạt mức production-grade, đặc biệt ở SBOM, signing, provenance, policy-as-code và quản lý secret. Thứ ba, browser-level E2E nên được mở rộng để bao phủ nhiều luồng UI hơn. Thứ tư, một số phần shared auth/contracts giữa service có thể tiếp tục được gom lại để giảm trùng lặp và tăng tính nhất quán.

Ngoài ra, nếu triển khai trong môi trường production thật, cần bổ sung thêm yêu cầu về backup/restore, disaster recovery, KMS, network policy, rate limiting nâng cao, logging retention, monitoring alert và quy trình vận hành sự cố.

8.4 Hướng phát triển

Trong các giai đoạn tiếp theo, DocVault có thể được phát triển theo các hướng sau:

1. **Hoàn thiện DevSecOps supply chain:** sinh SBOM, ký image bằng cosign, lưu provenance và kiểm tra artifact trước deploy.
2. **Policy-as-code:** bổ sung admission control cho Kubernetes để chặn image chưa scan, container chạy privileged hoặc manifest thiếu security context.
3. **Secret management:** tích hợp KMS/External Secrets, rotation runbook và secret scanning gate chặt hơn.
4. **Notification production:** thay notification sink bằng email/webhook/push hoặc message queue.
5. **Event-driven architecture:** bổ sung queue/event bus cho audit, notification và workflow asynchronous.
6. **Browser E2E:** mở rộng Playwright/Cypress để kiểm thử end-to-end ở lớp UI.
7. **Observability nâng cao:** tạo dashboard riêng cho business events, security alerts, latency, error rate và audit ingestion.
8. **Compliance pack:** chuẩn hóa export evidence theo kỳ, theo tài liệu, theo recommendation và theo security incident.

9. **AI governance:** phát triển các tính năng AI summarization/Q&A dựa trên guardrails metadata-only và content access policy.

8.5 Kết luận chung

DocVault cho thấy cách tiếp cận xây dựng một hệ thống quản lý tài liệu không chỉ dừng ở chức năng CRUD hoặc lưu file, mà đặt trọng tâm vào bảo mật, kiểm soát truy cập, audit và bằng chứng tuân thủ. Việc kết hợp ứng dụng microservices với quy trình DevSecOps giúp hệ thống có khả năng kiểm thử và triển khai có kiểm soát hơn. Mặc dù vẫn còn các hướng cần hoàn thiện, đề tài đã tạo được nền tảng đầy đủ để tiếp tục phát triển thành một hệ thống quản lý tài liệu bảo mật có tính thực tiễn cao.

Phụ lục A

Hướng dẫn chạy hệ thống

A.1 Yêu cầu môi trường

Môi trường phát triển DocVault cần các công cụ sau:

- Node.js 20 trở lên.
- pnpm 9 trở lên.
- Docker Desktop hoặc Docker Engine kèm Docker Compose.
- Git.
- Các port local trống cho gateway, microservices, frontend, Keycloak, PostgreSQL, MongoDB và MinIO.

A.2 Cài đặt phụ thuộc

Từ thư mục gốc repository, chạy:

```
pnpm install
```

A.3 Khởi động hạ tầng local

Hạ tầng local nằm trong `infra/docker-compose.dev.yml`. Lệnh chạy:

```
docker compose -f infra/docker-compose.dev.yml --env-file infra/.env up -d
```

Các thành phần được khởi động gồm PostgreSQL, MongoDB, MongoDB Express, MinIO, MinIO init job và Keycloak.

A.4 Chạy migration

Sau khi PostgreSQL đã sẵn sàng:


```
pnpm --filter metadata-service prisma:deploy
pnpm --filter audit-service prisma:deploy
```

A.5 Khởi động backend

Khuyến nghị chạy từng service ở terminal riêng theo thứ tự:

```
pnpm --filter metadata-service start:dev
pnpm --filter audit-service start:dev
pnpm --filter document-service start:dev
pnpm --filter notification-service start:dev
pnpm --filter workflow-service start:dev
pnpm --filter gateway start:dev
```

Các service mặc định:

- Gateway: <http://localhost:3000>
- Metadata-service: <http://localhost:3001>
- Document-service: <http://localhost:3002>
- Workflow-service: <http://localhost:3003>
- Audit-service: <http://localhost:3004>
- Notification-service: <http://localhost:3005>

A.6 Khởi động frontend

Frontend nên chạy trên port 3100 để tránh xung đột với gateway:

```
pnpm --filter web dev -- --port 3100
```

Truy cập:

```
http://localhost:3100
```

A.7 Tài khoản demo

Các tài khoản demo thường dùng:

- viewer1
- editor1
- approver1
- co1

- `admin1`

Mật khẩu demo nằm trong tài liệu hướng dẫn chạy nội bộ của repository và cần được thay đổi nếu triển khai ngoài môi trường dev.

A.8 Kiểm thử nhanh

Sau khi backend hoạt động:

```
pnpm test:e2e
```

Lệnh này kiểm tra các luồng chính: auth boundary, deny viewer, create/upload/submit của editor, approve của approver, download published document, audit của compliance và deny download của compliance.

Phụ lục B

Tóm tắt API contract

B.1 Nguyên tắc chung

Frontend gọi toàn bộ API thông qua gateway với base URL:

```
http://localhost:3000/api
```

Các request cần header:

```
Authorization: Bearer <keycloak_jwt>
```

Gateway xác thực JWT, kiểm tra role ở mức route và proxy request xuống service tương ứng. Các endpoint chi tiết tài liệu, workflow history, comments và ACL được policy-filter ở backend.

B.2 Nhóm endpoint chính

Bảng B.1: Tóm tắt endpoint chính

Chức năng	Endpoint	Vai trò chính
Login	GET /auth/login	Public
Current user	GET /auth/me	Authenticated
List documents	GET /metadata/documents	All roles
Create document	POST /metadata/documents	Editor, Admin
Document detail	GET /metadata/documents/:docId	Policy-filtered
Update metadata	PATCH /metadata/documents/:docId	Owner Editor, Admin
ACL list/upsert	GET/POST /metadata/documents/:docId/acl	Policy-filtered, Editor/Admin

Chức năng	Endpoint	Vai trò chính
Workflow history	GET /metadata/documents/ :docId/workflow-history	Policy-filtered
Upload file	POST /documents/:docId/upload	Editor, Admin
Authorize download	POST /metadata/documents/ :docId/download-authorize	Policy-filtered
Presign download	POST /documents/:docId/ presign-download	Policy-filtered
Stream file	GET /documents/:docId/ versions/:version/stream	Policy-filtered
Submit	POST /workflow/:docId/submit	Editor, Admin
Approve	POST /workflow/:docId/approve	Approver, Admin
Reject	POST /workflow/:docId/reject	Approver, Admin
Archive	POST /workflow/:docId/archive	Editor, Admin
Audit query	GET /audit/query	Compliance, Admin
Verify chain	GET /audit/verify-chain	Compliance, Admin
Security summary	GET /audit/security-summary	Compliance, Admin
Retention list	GET /metadata/retention/ documents	Compliance, Admin
Retention run	POST /metadata/retention/run	Admin
Evidence packet	GET /metadata/documents/ :docId/evidence-packet	Compliance, Admin

B.3 Compliance Officer rule

Compliance Officer được phép xem audit, retention, security summary và evidence packet. Tuy nhiên, vai trò này không được preview, stream, presign hoặc download file content. Quy tắc này là một trong các acceptance criteria quan trọng của đề tài.

B.4 Grant token

Download và preview grant token được cấp bởi metadata-service sau khi policy check thành công. Token có thời hạn ngắn và được document-service xác minh trước khi trả nội dung. Thiết kế này giúp tách bước quyết định quyền khỏi bước phát file, đồng thời tránh để frontend tự quyết định quyền.

Phụ lục C

Pipeline evidence

C.1 Tham số Jenkins khuyến nghị

Khi ứng dụng đã deploy lên EKS và web URL reachable, pipeline có thể chạy với các tham số:

```
RUN_ZAP=true
DEPLOY_TARGET_URL=http://<node-external-ip>:30006
ZAP_TARGET=http://<node-external-ip>:30006
RUN_ARGO_HEALTH_CHECK=true
KUBECONFIG_CREDENTIAL_ID=jenkins-argocd-kubeconfig
GITOPS_BRANCH=gitops-testing
```

C.2 Bảng chứng cần chụp

Bảng C.1: Danh sách pipeline evidence cần đưa vào khi nộp

Bảng chứng	Nội dung cần thể hiện
Jenkins build	Build number, branch, stage view xanh, timestamp.
Pre-build Security	Các stage SCA, Trivy FS, Unit Tests, SAST, Checkov, Terraform Validate pass.
Build & Scan	Service image được build và scan image.
Push & GitOps	Image tag được push, GitOps branch được cập nhật.
Argo CD	Các application chính Synced/Healthy.
Smoke test	GET / và GET /api/health pass.
ZAP report	zap_report.html và zap_report.json được archive.
Grafana	Pod health, CPU/RAM hoặc workload status.

Bảng chứng	Nội dung cần thể hiện
Loki	Query log namespace <code>docvault</code> .

C.3 Diễn giải khi bảo vệ

Khi trình bày pipeline, không nên chỉ nói “pipeline chạy xanh”. Cần giải thích pipeline chứng minh điều gì:

- SAST/SCA giúp phát hiện lỗi mã nguồn và dependency trước khi build image.
- Trivy/Checkov/Terraform validate giúp kiểm tra artifact và hạ tầng.
- Docker build và image scan giúp đảm bảo image tạo ra có thể triển khai và đã được quét.
- GitOps update giúp trạng thái deploy có lịch sử trong Git.
- Argo CD health chứng minh cluster đã reconcile về trạng thái mong muốn.
- Smoke test chứng minh app sau deploy reachable.
- ZAP DAST kiểm tra ứng dụng đang chạy và tạo report bảo mật động.

Phụ lục D

Danh sách hình ảnh minh họa

D.1 Ảnh UI đã sử dụng

Các ảnh trong `report/images/screen` được sử dụng cho phần triển khai ứng dụng và bằng chứng giao diện:

- `dashboard.png`: dashboard.
- `document.png`: danh sách và tổng quan tài liệu.
- `approvals.png`: phê duyệt.
- `audit.png`: audit log.
- `security.png`: security posture.
- `accessreview.png`: rà soát quyền truy cập.

D.2 Ảnh UI nên bổ sung

Để mô tả đầy đủ hơn các luồng chính của frontend, có thể bổ sung:

- Màn hình đăng nhập qua Keycloak.
- Form tạo tài liệu và upload file.
- Chi tiết tài liệu, version history và workflow timeline.
- Evidence Center và retention policy.
- Thông báo lỗi khi người dùng bị policy từ chối download.

D.3 Ảnh cần bổ sung trước khi nộp

Trước khi nộp bản cuối, nên bổ sung các ảnh pipeline/evidence sau:

- Jenkins stage view của build mới nhất.
- SonarQube project dashboard.

- Dependency Check report.
- Trivy image scan log/report.
- Checkov/Terraform validate log.
- Docker Hub hoặc registry image tags.
- GitOps commit cập nhật Helm values.
- Argo CD applications Synced/Healthy.
- ZAP HTML report summary.
- Grafana workload dashboard.
- Loki query log namespace `docvault`.

Khi chụp ảnh, cần che hoặc loại bỏ token, secret, kubeconfig, IP nhạy cảm và thông tin tài khoản không dùng cho demo công khai.

Tài liệu tham khảo

- [1] National Institute of Standards and Technology. “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities”, visited on June 14, 2026. address: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [2] OWASP Foundation. “OWASP DevSecOps Guideline”, visited on June 14, 2026. address: <https://owasp.org/www-project-devsecops-guideline/>
- [3] Keycloak. “Keycloak Documentation”, visited on June 14, 2026. address: <https://www.keycloak.org/documentation>
- [4] Argo CD Project. “Argo CD: Declarative GitOps CD for Kubernetes”, visited on June 14, 2026. address: <https://argo-cd.readthedocs.io/en/stable/>
- [5] SonarSource. “SonarQube Server Documentation”, visited on June 14, 2026. address: <https://docs.sonarsource.com/sonarqube-server>
- [6] Checkov Project. “Checkov Documentation”, visited on June 14, 2026. address: <https://www.checkov.io/1.Welcome/What%20is%20Checkov.html>
- [7] Aqua Security. “Trivy Documentation”, visited on June 14, 2026. address: <https://trivy.dev/docs/latest/guide/>
- [8] ZAP by Checkmarx. “ZAP Baseline Scan”, visited on June 14, 2026. address: <https://www.zaproxy.org/docs/docker/baseline-scan/>